UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Multimedia Communications

Adaptive Streaming

20/05/2026

Lecture Overview

1. **Protocol Evolution**: Comprehend the technical transition from traditional network-driven "Push" architectures (RTP over UDP) to modern client-driven "Pull" adaptive infrastructures running over HTTP.
2. **Modern Transport Optimization**: Analyze the concrete impact of QUIC and HTTP/3 frameworks on high-bitrate multimedia streaming performance.
3. **Analytical System Modeling**: Formulate and evaluate the continuous-time mathematical fluid model characterizing playout buffer behavior.
4. **User Experience Engineering**: Quantify Quality of Experience (QoE) metrics and study the design trade-offs inside Adaptive Bitrate (ABR) decision loops.

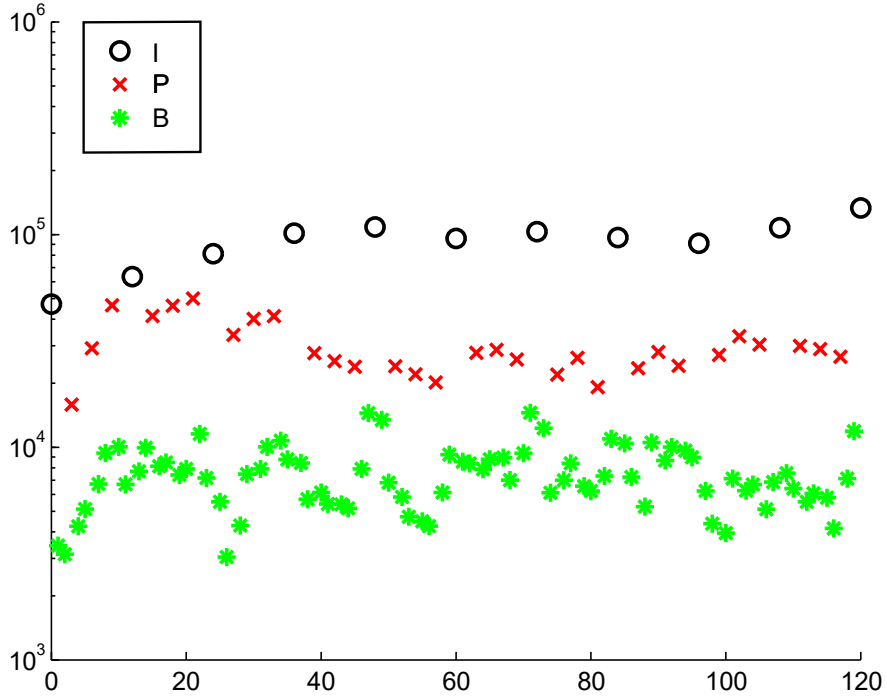
Classification of Multimedia Services

The table below categorizes multimedia traffic based on technical requirements and tolerance to network impairments.

Service Category	Real-world Examples	Critical Requirement	Delay Tolerance
Bulk Transfer	Social Media Photos, Messaging	Data Integriyt (No Loss)	**High (Seconds)**
Video On-Demand	Netflix, YouTube, Disney+	Continuity (No Jitter)	**Medium (1-3s startup)**
Streaming Live	Twitch, DAZN, YouTube Live	Scalability and Recency	**Low (< 10s)**
Real-time Interattivo	Teams, Zoom, WhatsApp Call	Ultra-low Latency	**Critical (< 150ms)**
Cloud Gaming / VR	GeForce Now, Meta Quest	Interatction + Bitrate	**Extreme (< 50ms)**

Video Communication through networks

- We know that a video source (uncompressed) emits bits at a *constant rate*
  - Example: uncompressed YCbCr HD video requires 1920×1080 pixels, with a typical frame rate of 50 Hz. Given 12 bits per pixel (4:2:0 sampling scheme), we end up with 1.244 Gbps
- After compression this rate is much reduced
  - Typical rates for this example are in the range $5 \div 20$ Mbps
- On the other hand, the coding rate is *no longer constant*
  - I frames are much "heavier" than P and B
- The compressed bitrate stabilizes and can be considered roughly constant only when averaged at the **Group of Pictures (GOP)** level.

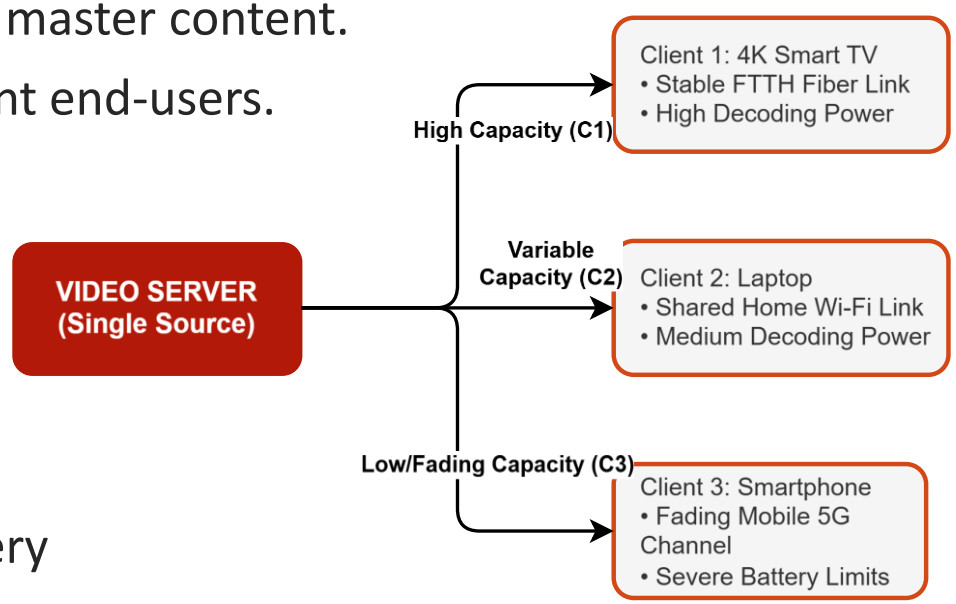


Frame size [bits] in a rate-controlled H.264 encoder

Video Content Distribution Challenges

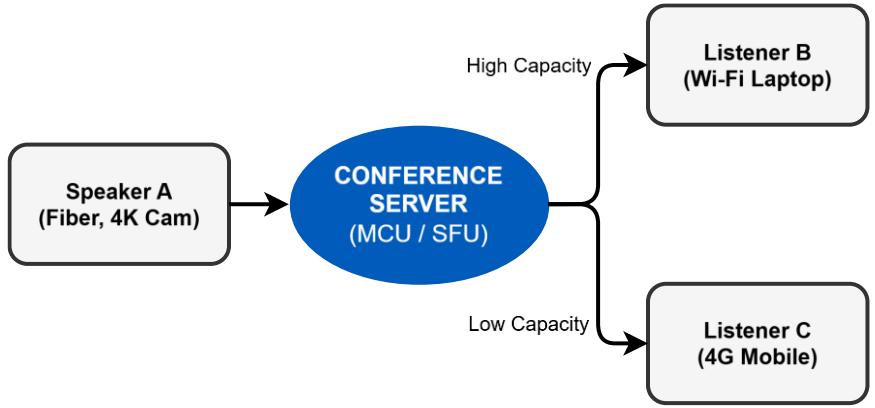
The Distribution Paradigm: One-to-Many Architecture

- **Single Source**: A central video server hosting and transmitting the master content.
- **Mass Destinations**: Millions of concurrent, completely independent end-users.
- **The Four Dimensions of User Heterogeneity**:
- *Network Paths*: Highly disparate link capacities, asymmetric propagation delays, and routing behaviors.
- *Temporal Dynamics*: Time-varying channel conditions, dominated by fast fading and shadowing in wireless/mobile links.
- *Hardware Constraints*: Disparate local memory sizes, varying battery lifespans, and available hardware decoding power.
- *Display Formats*: Massive span in form factors, stretching from low-resolution smartphone screens up to premium 4K/8K TV sets.



The Conversational Paradigm

- **One-to-One**: Video calls
- *Challenge*: The channel capacity fluctuates constantly. The encoder must adapt the bitrate "live" with zero-delay buffers.
- **Few-to-Few (Multipoint)**: Group conferencing (
- *Central Node*: Calls are routed through a central Selective Forwarding Unit (SFU) or Multipoint Control Unit (MCU).
- **The Live Engineering Bottleneck**:
- Exactly like streaming, the central node faces a highly **heterogeneous** group of receivers.
- *The Difference*: We cannot rely on pre-encoded video files. The system must encode and distribute live video satisfying all different user capacities under extreme **sub-150ms** latency constraints



The Coding Rate Bottleneck

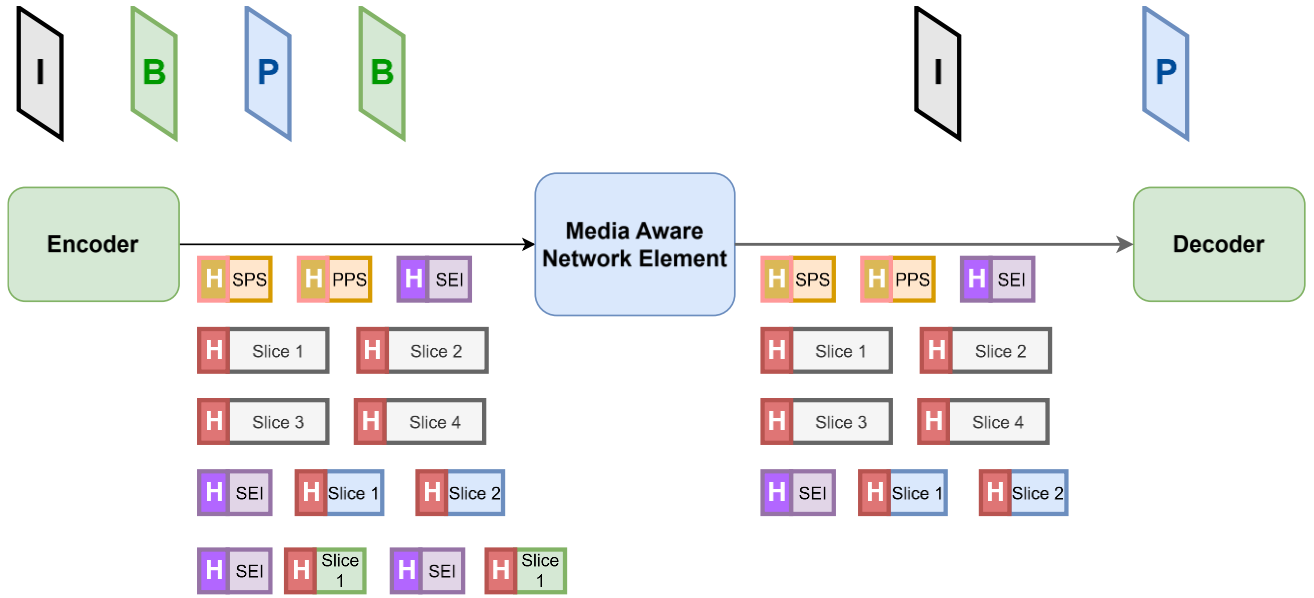
- **The Operational Constraint**: To ensure continuous playout or fluid interaction, the video bitrate R_C must satisfy the channel capacity limit C_k for user k:

$$R_C \leq C_k$$

- **The Symmetrical Dilemma**: Since each client sees a completely different C_k, a single static choice of R_C harms the system performance:
  - **The Cliff Effect**: If $R_C > C_k$, packets accumulate in network buffers, causing delays, drops, and severe playback freezes .
  - **The Leveling Effect**: If $R_C = \min_k C_k$, high-capacity users are forced to watch low-quality streams, wasting their channel potential
- **Conclusion**: The static delivery model must be replaced by an **adaptive coding rate** framework.

Compression Fragility & Network Friendliness

- **The Open Internet: Blind IP Routing**
- Highly compressed video relies on strict spatial/temporal prediction and CABAC contexts.
- Standard IP routers drop packets blindly during congestion, causing catastrophic spatio-temporal error propagation

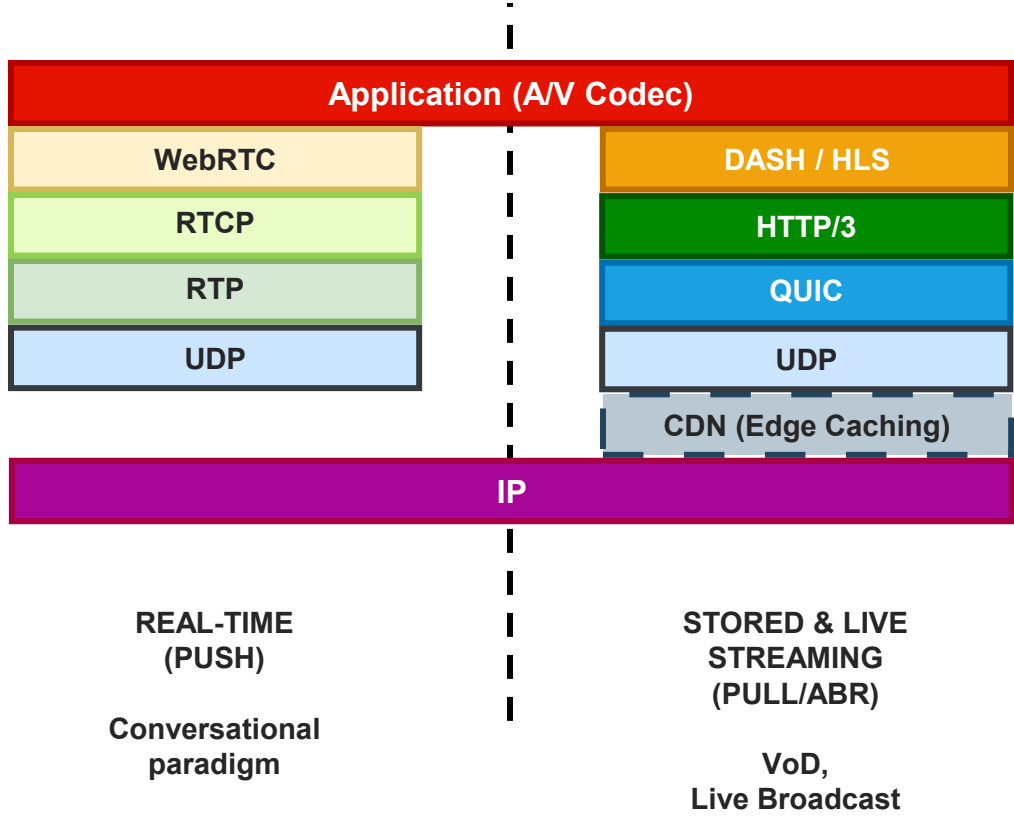


The Managed Solution: Media Aware Network Elements (MANEs)

- Modern codecs (H.264 onwards) separate compression (VCL) from transport (NAL) .
- In managed domains (e.g., **CDN Edge nodes**, **5G Mobile Gateways**), specialized middleboxes called MANEs can inspect the NAL header .
- **Smart Dropping**: MANEs safely discard Non-Reference B-frames during congestion, protecting critical data (SPS, PPS, IDR) without needing to decode the video.

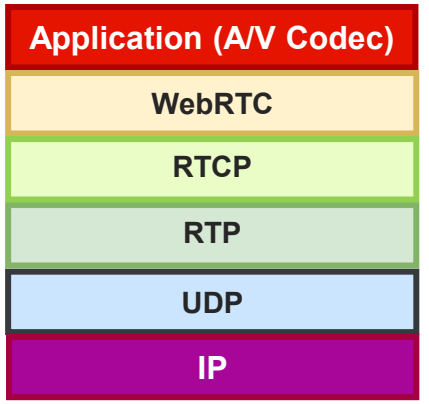
Modern Multimedia Stacks: Push vs. Pull

- **The Real-Time (Push) Paradigm**:
  - *Core Objective*: Minimize absolute interactive latency (<150 ms).
  - *Control Model*: Server-driven execution; content is immediately pushed to the client upon generation.
  - *Dominant Stack*: Driven by WebRTC and RTP mechanisms over a lightweight UDP base
- **The Adaptive Streaming (Pull) Paradigm**:
  - *Core Objective*: Maximize visual Quality of Experience and handle large-scale global concurrency.
  - *Control Model*: Client-driven execution; the receiver explicitly requests discrete media blocks based on local status.
  - *Dominant Stack*: Convergence toward HTTP/3 running over user-space QUIC transport engines



The Real-Time Stack: WebRTC, RTP, and UDP

- **UDP**: bare-bone protocol, only offers MUX/DEMUX (via sockets), eliminates connection handshakes and retransmission delays. Trades perfect data integrity for absolute minimum
- **RTP (Real-time Transport Protocol):** adds to UDP **sequence numbers** (to detect lost or out-of-order packets) and **timestamps** (for A/V synchronization and jitter management)
- **RTCP (RTP Control Protocol):** periodically sends QoS metrics (packet loss rate, RTT) back to the sender, enabling dynamic R_C adaption
- **WebRTC (Web Real-Time Communication):** wraps RTP/RTCP into a browser-native framework, enforcing mandatory encryption and solving connectivity problems (NAT/Firewalls)
- **Use cases:** Videocall, cloud gaming, remote control



REAL-TIME (PUSH)

Conversational paradigm

One-to-Many Real-Time Communications: the Scalable Video Coding (SVC) Paradigm

Video call: how can the encoder provide a different coding rate for each destination?

The SVC Concept: *"Encode Once, Decode Many"*.

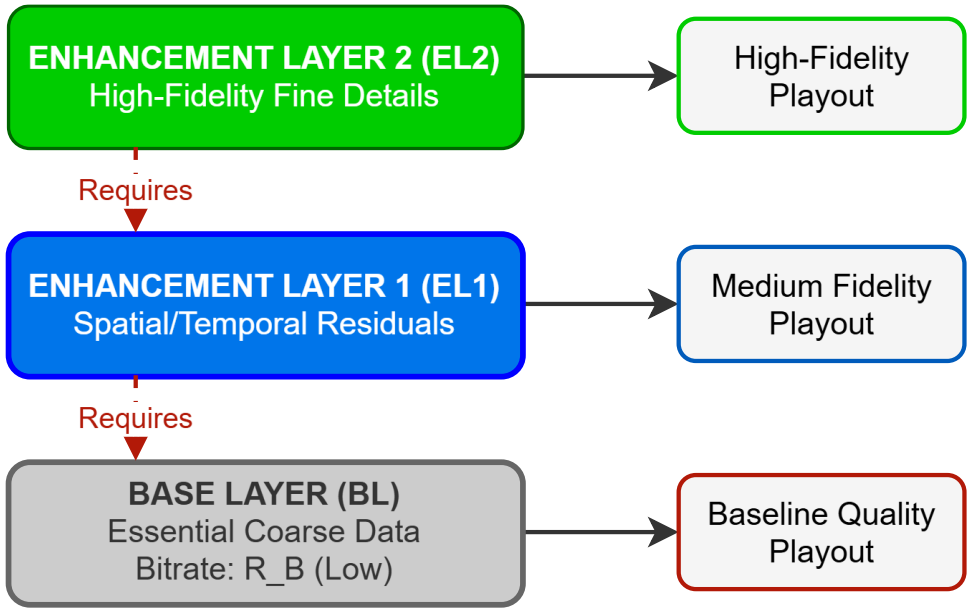
- Compresses the source video into a **single, unified bitstream** layered by importance.

The Bitstream Hierarchy:

- **Base Layer (BL)**: Contains the essential low-bitrate data (R_B).
- Highly protected, guarantees a baseline minimum quality: $R_B \leq \min_k C(k)$
- **Enhancement Layers (EL)**: Multiple progressive refinement layers (N-1) containing details lost at the base layer

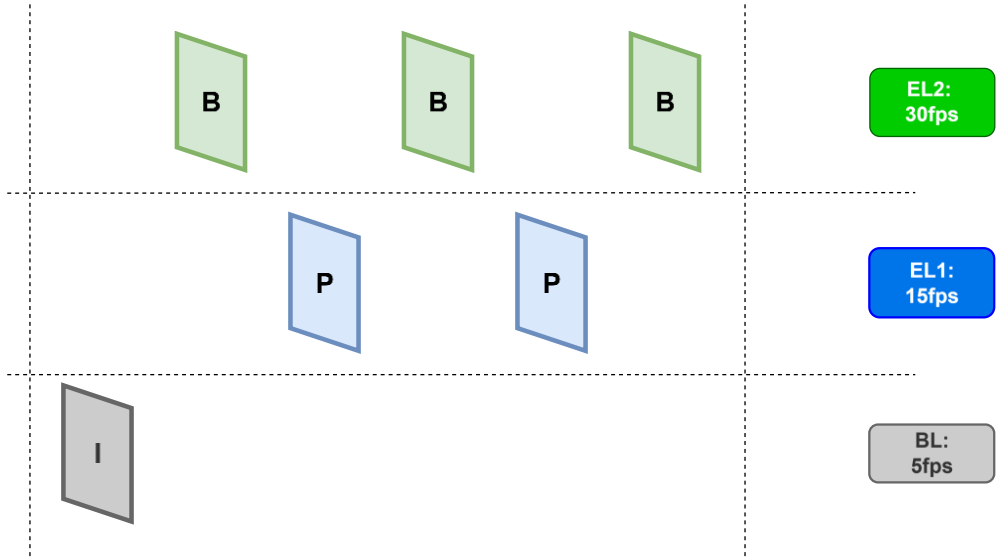
Independent Transmission:

- Each layer behaves as an independent network payload sub-stream.
- Enhancement layers provide higher fidelity, spatial resolution, or framerate, but are **entirely useless** without the Base Layer.

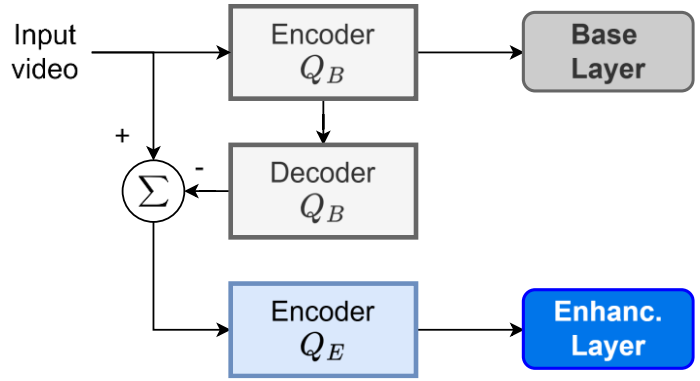


Types of Scalability

- **Time scalability**: can be obtained by suitably deciding the temporal prediction structure



- **Quality scalability:** a quality version of the video is encoded in the BL and used as prediction of the ELs; **or** use progressive coding (e.g., bitplanes)
- **Space scalability:** similar to QS, but the base layer is low-resolution; **or** use multi-resolution transforms
- A layer can use a combination of scalability types; this is signaled in the NAL header (ID)



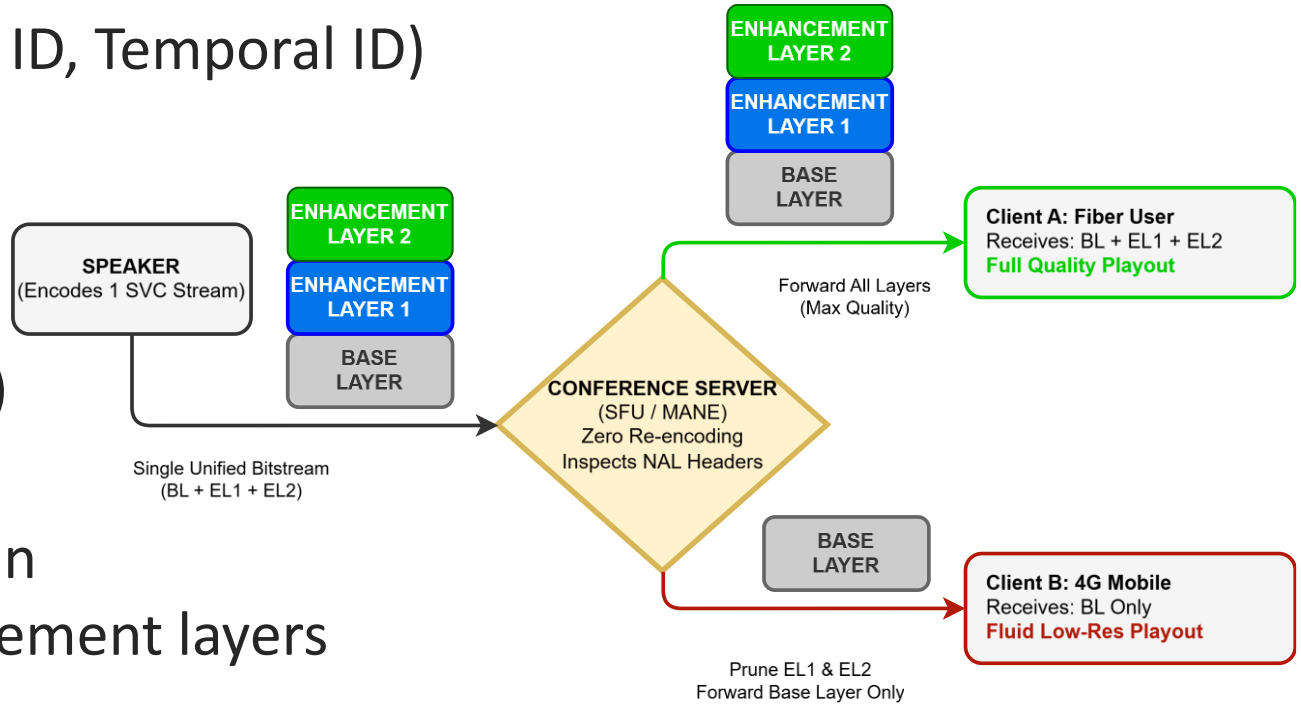
SVC Infrastructure & Selective Forwarding

In multipoint conferencing, the speaker send the SVC coded video to a server called *Selective Forward Unit* that acts as a MANE

- It inspects the NAL header metadata (Layer ID, Temporal ID) without touching the VCL payload

Smart Layer Pruning:

- *High-Capacity Path*: Forwards the complete layered bitstream (BL + EL1 + EL2)
- *Low-Capacity Path*: Instead of dropping random packets blindly, the SFU performs an *intelligent NAL-level drop*, stripping enhancement layers to forward **only the Base Layer**.



- According to the number of layers, the adaption can be more or less fine-grained
- **Result**: Adaptation with zero server re-encoding overhead and zero error propagation.

Why SVC cannot be used for Web-scale streaming

The Infrastructure & Cost Explosion:

- In one-to-few, real-time video, a single Media-Aware Network Element (MANE/SFU) is computationally viable and cost-confined.
- At web-scale (millions of users), scaling SVC would require deploying **thousands of interconnected MANEs/routers** across the entire network topology to filter layers per-user. This destroys the economic viability and the stateless routing principle of the Internet

- SVC introduces a heavy **syntactic and signaling overhead** (10% to 25% higher bitrate compared to single-layer streams at identical visual quality) due to inter-layer prediction data
- SVC splits video into **hierarchically dependent layers** (Base + Enhancement). CDNs cannot map these efficiently to static, atomic URI caches. If a Base Layer segment drops, all associated enhancement layers instantly become un-decodable garbage.
- **The Client Hardware Barrier**: Layered decoding spikes computational complexity. Mass-market consumer hardware chipsets (Smart TVs, cheap mobile SOCs) lack native ASIC blocks for SVC due to manufacturing costs

Why HTTP works well for Video Streaming?

The Firewall & NAT Traversal Advantage:

- Legacy streaming protocols (RTSP/RTP over UDP) use random dynamic ports routinely blocked by corporate and home firewalls.
- HTTP leverages standard web ports (**80/443**), ensuring universal, unrestricted network traversal.

Stateless Server Scalability:

- The origin server does not maintain connection states, playout clocks, or client-specific session timers.
- It operates as a highly optimized, stateless file dispenser, radically reducing CPU and memory overhead.

Native Caching Integration:

- Slicing video into standard HTTP objects natively integrates with existing internet infrastructure
- Standard web proxies, edge caches, and **Content Delivery Networks (CDNs)** replicate media files seamlessly without requiring specialized video-aware software.

… but classic web traffic runs on **TCP**, inheriting its rigid state machine

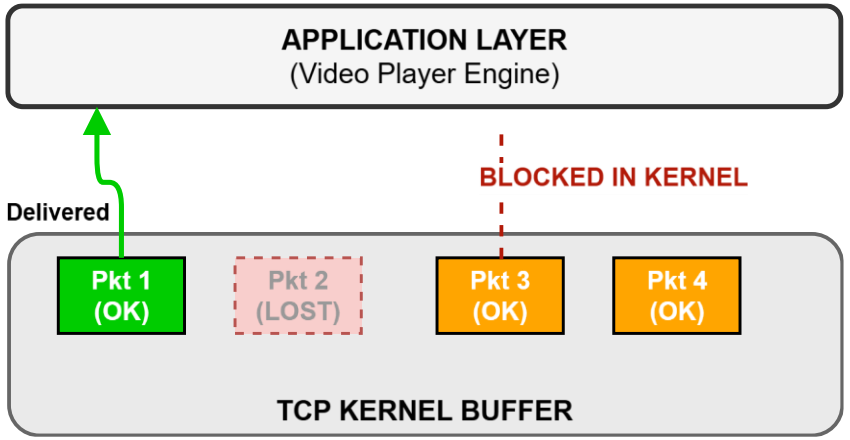
The Latency Paradox & TCP Limitations

The Transport Dilemma:

- HTTP streaming over classic networks relies on the Transmission Control Protocol (**TCP**).
- TCP guarantees **absolute reliability**, which forces an aggressive penalty on real-time multimedia delivery.

Head-of-Line (HOL) Blocking:

- TCP treats data as a single, sequential, unbroken queue of bytes.
- If an intermediate packet (e.g., Packet 2) is dropped by the network, the operating system kernel **freezes the entire queue**.
- Subsequent packets (Packets 3, 4, etc.) are held back in kernel memory, even if they arrived perfectly.



The Playout Impact:

- The application layer receives zero data while TCP waits for the slow retransmission timeout and repair.
- This transport freeze causes sudden **buffer depletion** and immediate application stalls.

The Generational Leap: QUIC and HTTP/3

Shifting the Transport Base:

- HTTP/3 abandons TCP completely, shifting its foundation to **UDP**.
- Reliability and congestion control are moved from the OS kernel to user-space via **QUIC**.

True Stream Isolation:

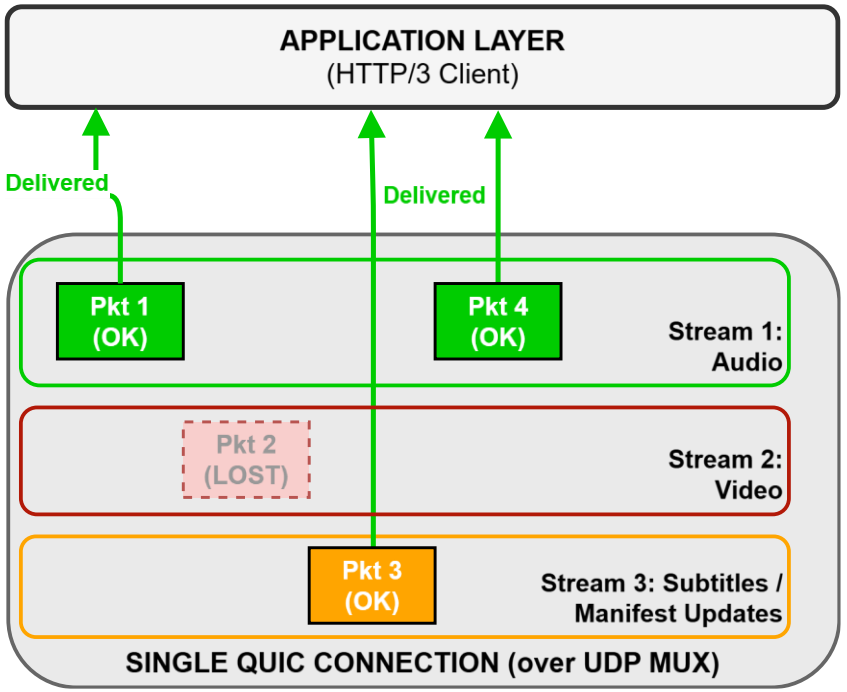
- QUIC treats different data components as logically **independent streams**.
- If a packet in Stream 2 is lost, only Stream 2 pauses. Other streams (e.g., subsequent video frames or audio) continues delivery with **zero HOL blocking**.

0-RTT Connection Handshake:

- Combines transport and cryptographic (TLS 1.3) handshakes into a single round-trip, minimizing startup delays.

Connection Migration:

- Connections are tracked via unique **Connection IDs**, not IP/Port pairs.Playout survives instant Wi-Fi to 5G cellular handovers without disconnection.



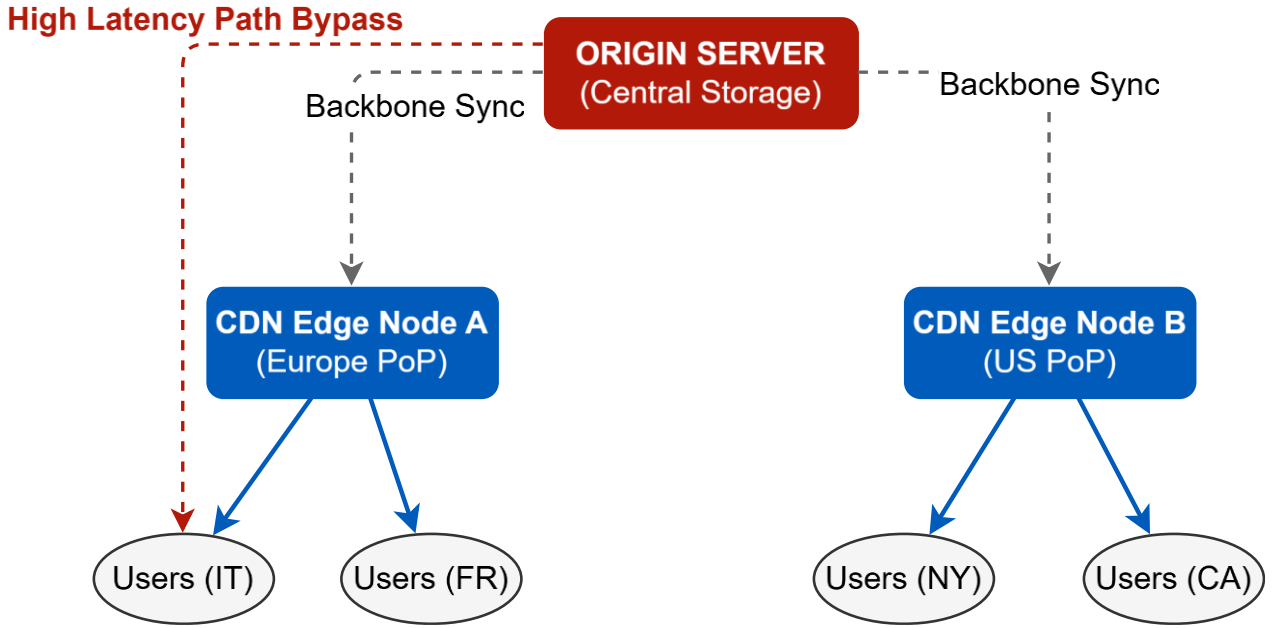
Content Delivery Networks (CDNs)

The Geographical Obstacle:

- Centralized content deployment introduces high propagation delay (proportional to the distance) and systemic core bottlenecks.

CDN Edge Architecture:

- A global grid of geographically distributed proxy servers, known as **Edge Nodes** or **Points of Presence (PoPs),** deployed directly inside local ISP networks.



The Three Core Pillars of a CDN:

- **Edge Caching**: Replicating and storing stateless HTTP video chunks as close to the end-user as physically possible.
- **Request Routing**: Utilizing intelligent DNS or Anycast frameworks to transparently redirect the client to the closest Edge Node.
- **Server Consolidation**: Radically reducing the origin server's processing load, absorbing massive global traffic spikes.

UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Adaptive Bitrate Streaming

The Architecture of Adaptive Streaming

The Industrial Standard: **MPEG-DASH**

- **D**ynamic **A**daptive **S**treaming over **H**TTP (ISO/IEC 23009-1).
- The first international standard for HTTP-based adaptive bit-rate streaming.

The Core Architectural Shift:

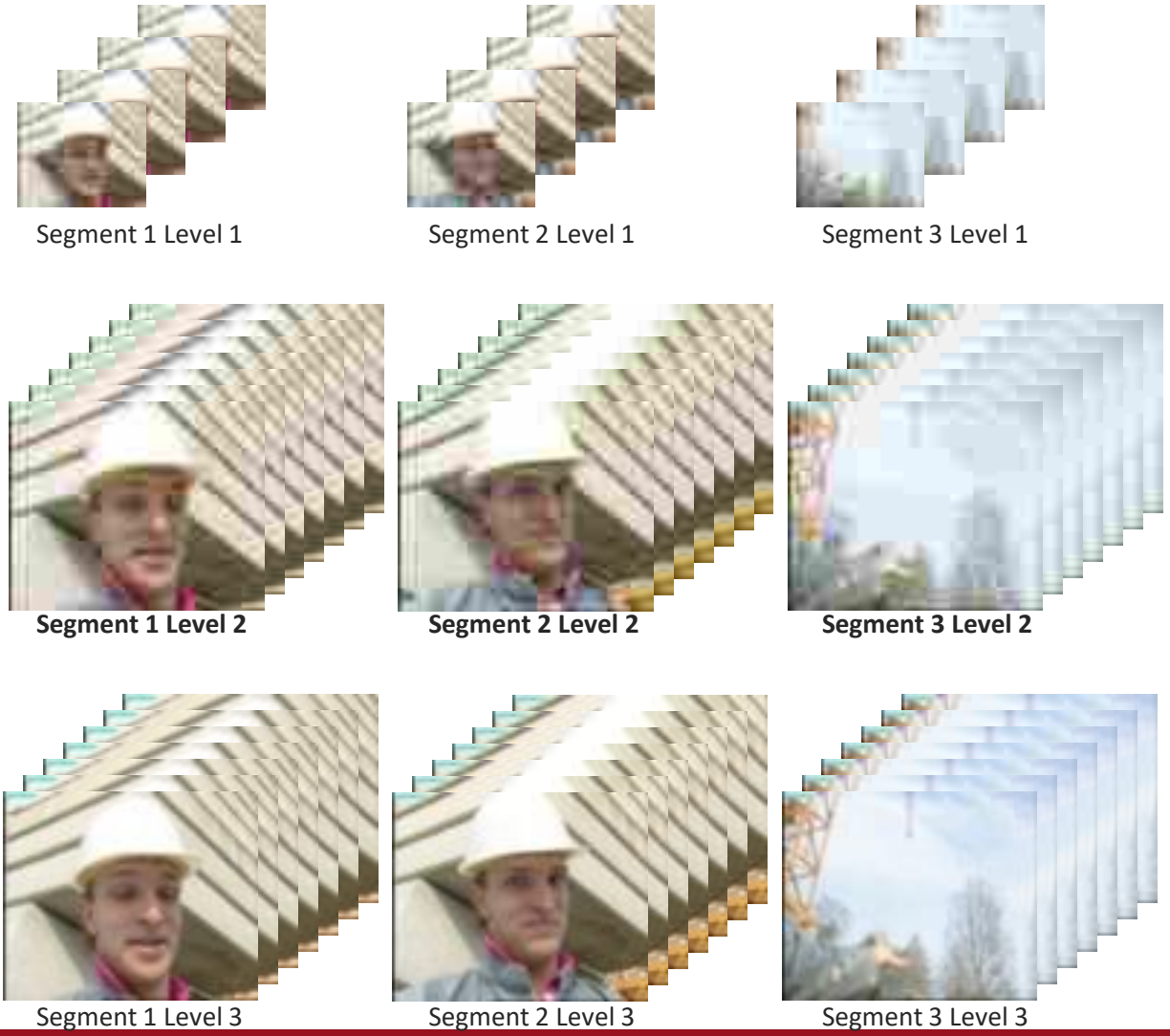
- **Dynamic**: The video quality adapts on-the-fly during playback based on real-time network conditions.
- **Adaptive**: The adjustment logic is entirely client-driven (intelligence moved to the network edge).
- **Streaming over HTTP**: Leverages standard web servers and ubiquitous Content Delivery Networks (CDNs) as passive, stateless file dispensers.

Video segments and representations (levels)

Video sequence divided into N segments of duration T_S

- Segment n is encoded at different resolution/quality combination, resulting into K **representations** or **levels**
- Level $k \in \{1,2,\dots,K\}$ is characterized by:
  - The coding rate $\boldsymbol{R_C(k)}$
  - The resolution Res_k
  - The frame rate F_k
- These values do not vary from a segment to the other, but vary across levels
- These information are available in the **Manifest File**

Level	Bitrate	Resolution	Frame rate
1	200 kbps	270×480	25 fps
2	500 kbps	540×960	50 fps
3	1 Mbps	540×960	50 fps
…	…	…	…

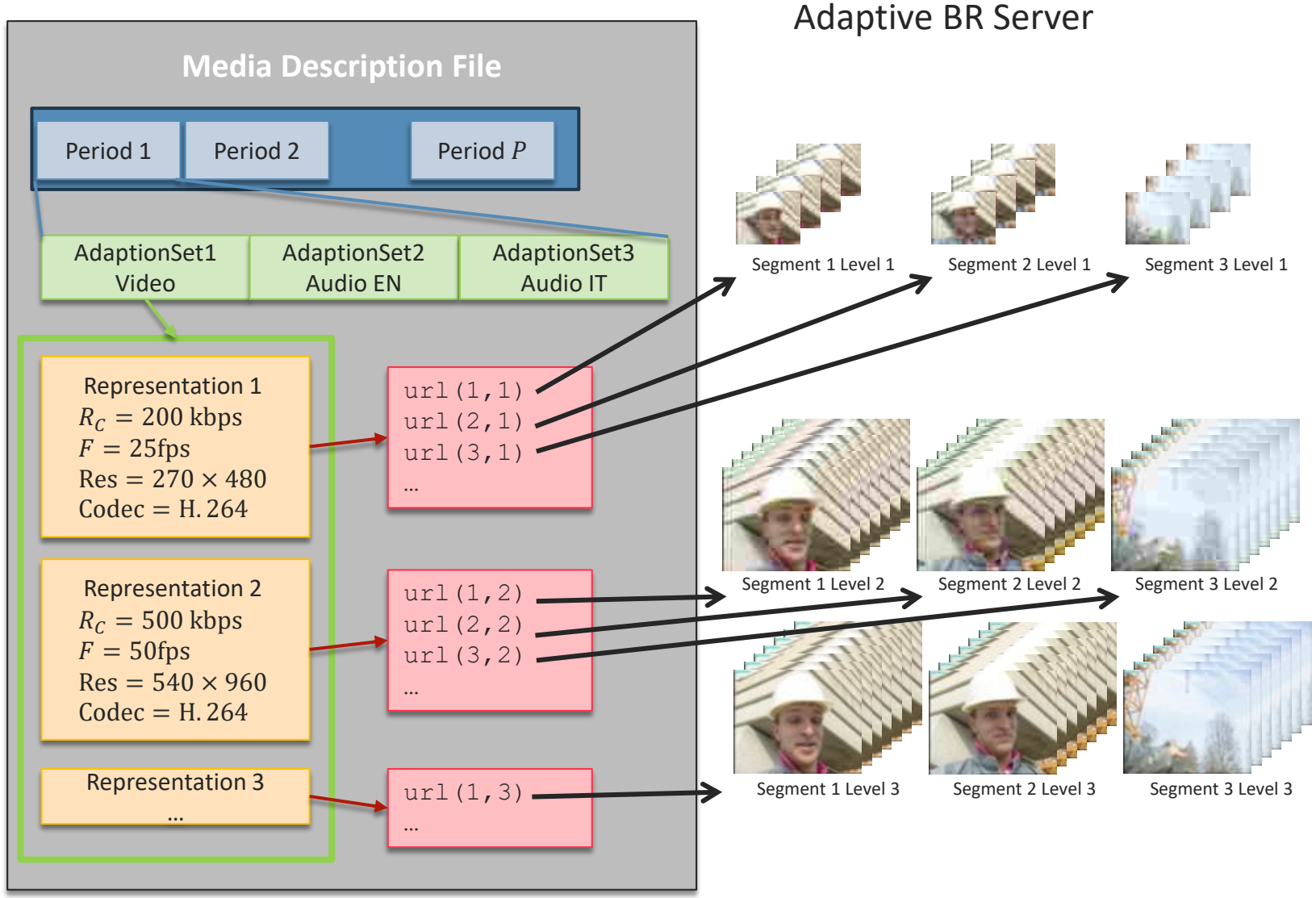


The Media Presentation Description (MPD) Manifest File

The Media Presentation Description is a structured **XML document** that profiles the entire multimedia asset hosted on the server.

The Hierarchical Architecture:

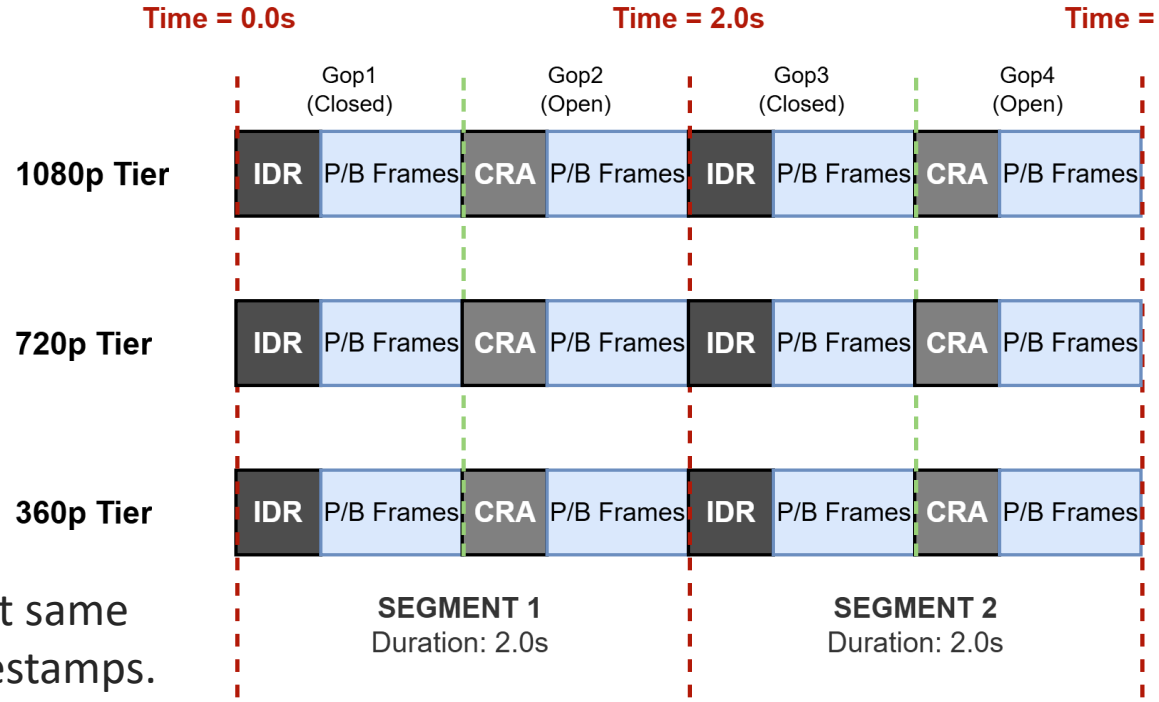
- **Period**: High-level temporal segments of the timeline (e.g., separating main content from ad insertion).
- **AdaptationSet**: Groups alternative media tracks of the same type (e.g., separating the Video track from the Audio track).
- **Representation**: A specific quality tier inside an AdaptationSet (e.g., 1080p at 5 Mbps vs. 720p at 2.5 Mbps). Profiles codecs, frame rates, and resolutions.
- **Segment**: The lowest logical node containing the exact HTTP URLs or byte-range templates for individual media chunks.



Video Segment Generation

The Switching Requirement:

- Bitrate adaptation can only execute at **each new segment**
- Every **segment** must start with a **closed GOP,** i.e. a GOP beginning with an **IDR frame**
- Open GOPs (beginning with **CRA**) are forbidden **at segment boundaries**: no frame inside Segment $N + 1$ can reference pixels back in segment N.



Cross-Representation Alignment: Segments across *all* parallel quality representations (360p, 720p, 1080p) must have the exact same temporal duration (T_S, e.g., 2.00 seconds) and identical IDR timestamps.

The Segment Duration Trade-Off:

- *Short Segments (1-2s)*: High granularity, rapid adaptation to fast network drops, but lower compression efficiency due to frequent, heavy IDR blocks.
- *Long Segments (6-10s)*: Better compression efficiency (fewer IDR overheads), but slow network reaction times and higher risk of buffer starvation.

Live Streaming vs. Stored Video Paradigms

Stored Video (On-Demand / VOD):

- Video generation is completely **asynchronous** to transmission.
- Media content is pre-encoded, pre-segmented, and statically cached on CDNs months in advance. Latency is non-critical.

Live Streaming (The Context Shift):

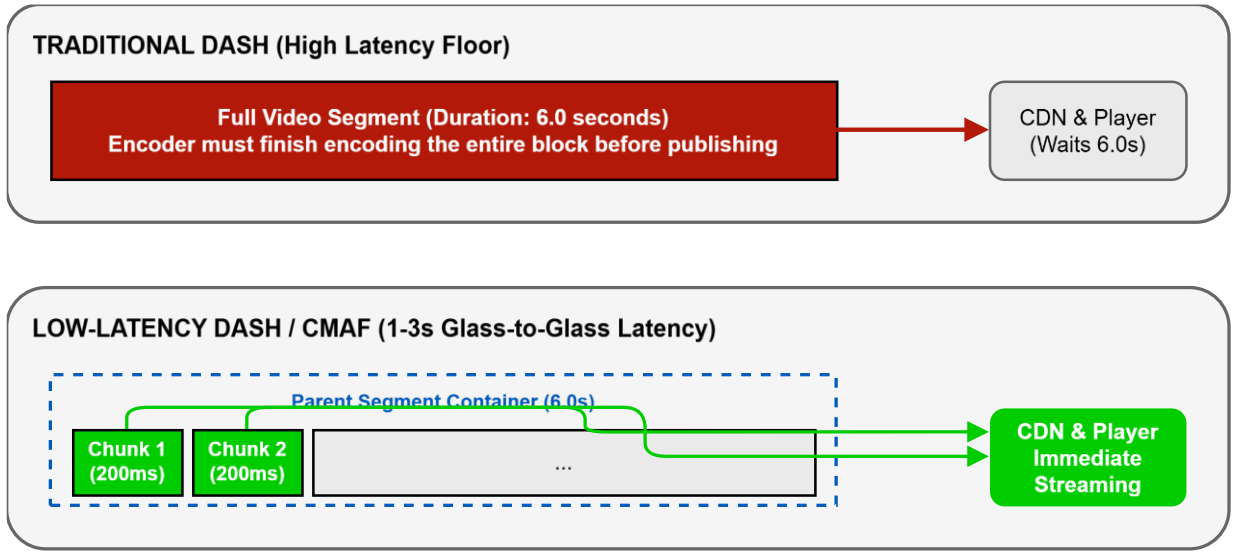
- Video generation is **simultaneous and contextual** to transmission.
- Segments are created dynamically, on-the-fly, by a real-time encoder.

Why DASH Fits Live (Relaxed Constraints):

- Unlike conversational Real-Time stack (WebRTC, <150ms), live streaming allows relaxed interaction delays.
- Audiences tolerate a 10-30s broadcast lag behind the physical event. This allows DASH to preserve its scalability and caching advantages.

Low-Latency Streaming Frameworks

- **The Wait-to-Encode Bottleneck**:
- Traditional DASH/HLS latency is strictly tied to the segment length: a server cannot publish or cache a 6-second segment until the encoder has completely finished writing it.
- **CMAF Sub-Segmentation**:
- The **Common Media Application Format (CMAF)** breaks down the structural segment into micro-units called **CMAF Chunks** (200ms - 500ms).
- Chunks do not require independent IDR frames; they share the single parent segment's anchor IDR.



- **Pipelined Delivery via HTTP CTE**:
- Leverages **HTTP/1.1 and HTTP/2 Chunked Transfer Encoding (CTE)**.
- As soon as the encoder finishes compressing a 200ms chunk, the HTTP origin server immediately streams it down the CDN pipeline, before the parent segment is closed.
- **The Latency Target**:
- Collapses glass-to-glass latency down to **1-3 seconds**, achieving parity with traditional cable TV/broadcast without sacrificing commodity HTTP caching infrastructure.

Network Metrics: Throughput and Capacity

- **Link Bitrate R**: the transmission rate supported by the **physical layer** interface (measured in bits/second).
- **Instantaneous Throughput $S(t)$**: The actual, time-varying rate at which data bytes successfully arrive at the **application layer** at a specific time t:

$$S(t) = \frac{dD(t)}{dt} \leq R$$

where $D(t)$ represents the cumulative data successfully received up to time t.

It is smaller than R because of the headers, the losses, the flow and congestion control

- **Average Throughput $\bar{S}$**: Measured over a longer window T to smooth out network bursts and packet drops:

$$\bar{S} = \frac{1}{T}\int_0^T S(t)\,dt = \frac{D(T)}{T}$$

Nodal Delay and End-to-end Delay

The total End-to-End Nodal Delay d_{e2e} of a packet is the sum of delay factors at each network hop k:

$$d_{e2e} = \sum_{k} d_{\mathrm{proc}}(k) + d_{\mathrm{queue}}(k) + d_{\mathrm{tx}}(k) + d_{\mathrm{prop}}(k)$$

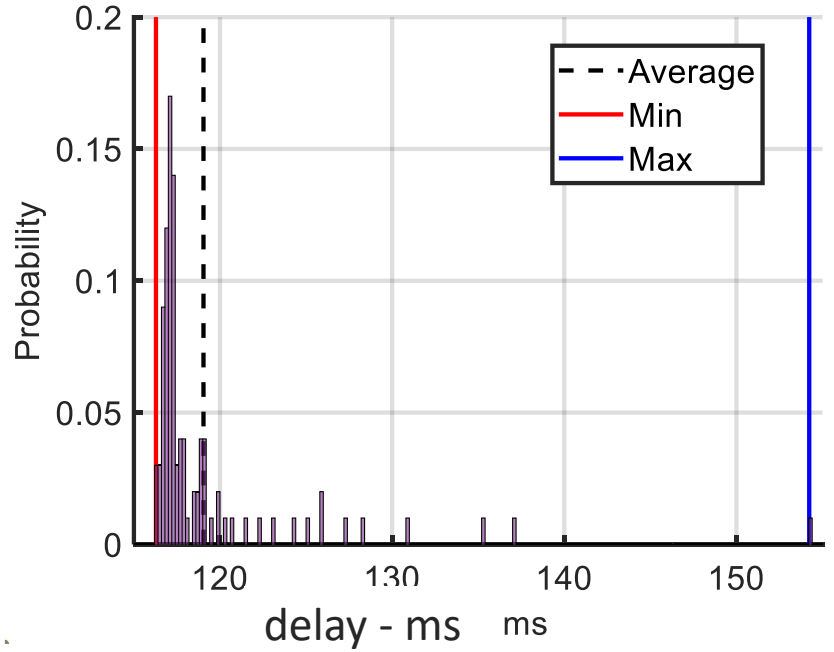
- **Processing delay $d_{\mathrm{proc}}(k)$**
- **Queuing delay $d_{\mathrm{queue}}(k)$**
- **Transmission delay $d_{\mathrm{tx}}(k)$**
- **Propagation delay $d_{\mathrm{prop}}(k)$**

Nodal Delay Breakdown

- **Processing $d_{\text{proc}}(k)$**: Time spent by routers checking bit-level errors, reading packet headers, deciding the output link
- **Queuing $d_{\text{queue}}(k)$** : Time a packet spends waiting in a router's buffer due to link congestion. *Dynamic and highly variable.*
- **Transmission $d_{\text{tx}}(k)$**: Time required to push all the packet's bits into the physical link ($d_{\text{tx}}(k) = L/R$, where L is packet length and R is link rate).
- **Propagation $d_{\text{prop}}(k)$** : Time it takes for a bit to travel through the physical medium at the speed of light $d_{\text{prop}}(k) = d/s$ where d is distance and s is propagation speed).
- **The Latency limit**: Increasing Transmission Rates R drops d_{tx}, but **does not reduce d_{prop}** or **d_{queue}**. Distance and congestion remain the structural bottlenecks of live interactive video.

Jitter

- The per-packet delay is determined by many different phenomena
- We are interested to the average delay and to its **variance,** called **jitter**
- High jitter means that the delay can sometimes be much different than its average: how to cope with these occasional peaks in delay?



Example of per-packet delay in an end-to-end internet connection. Even though most of the packets experience a delay less than 120ms, for some it can be as large as 150ms

The Playout Buffer: The Client's Defensive Reservoir

- To absorb the delay variability (jitter), the client has a **Playout Buffer**
- It isolates the stochastic, unpredictable packet arrivals from the network from the deterministic frame decoding process

- The main parameter is the buffer size at time t, referred to as $B(t)$
- It is measured in **seconds** and represent how much video data has been received but not played yet

- It is the *Client's Defensive Reservoir*: if the network fails at time t, the playout can continue for $B(t)$ further seconds

Playout Buffer Dynamics

- We use a continuous-time, continuous-valued model for $B(t)$ (fluid dynamics framework)
- At the beginning, the buffer is empty $B(t)$
- Typically, when the first bits of the first segment are received, the playout does not start immediately. Rather, the client builds up a reservoir of L segments
- When $B(t) = L \cdot T_S$ the playout starts, and the buffer can grow or shrink according to the network conditions and to the requested R_C
- **Buffer Starvation: $B(t) = 0$.** The tank empties completely, triggering an immediate playout freeze (rebuffering stall).
- The playout is not resumed until M segments are received: after a rebuffering stall, $B(t) = M \cdot T_S$
- Typically, $L \geq M$ because users are more tolerant to initial delay than to stalls

Playout Buffer Dynamics

- We model the buffer dynamics through the input and the output flow rates: $\frac{dB}{dt} = f_{\text{in}} - f_{\text{out}}$
- **Input Flow Rate f_{in}**: The seconds of video content received per real-time second.
  - The client receives $\bar{S}T$ bits in an interval of duration T
  - This represent a playout duration of $D(T) = R_C T$
  - Therefore, the average input flow rate in the interval is $\frac{D(T)}{T} = \frac{\bar{S}}{R_C}$
  - The instantaneous flow rate is $\lim_{T \to 0} \frac{D(T)}{T} = \frac{S(t)}{R_C}$
- Driven by network conditions

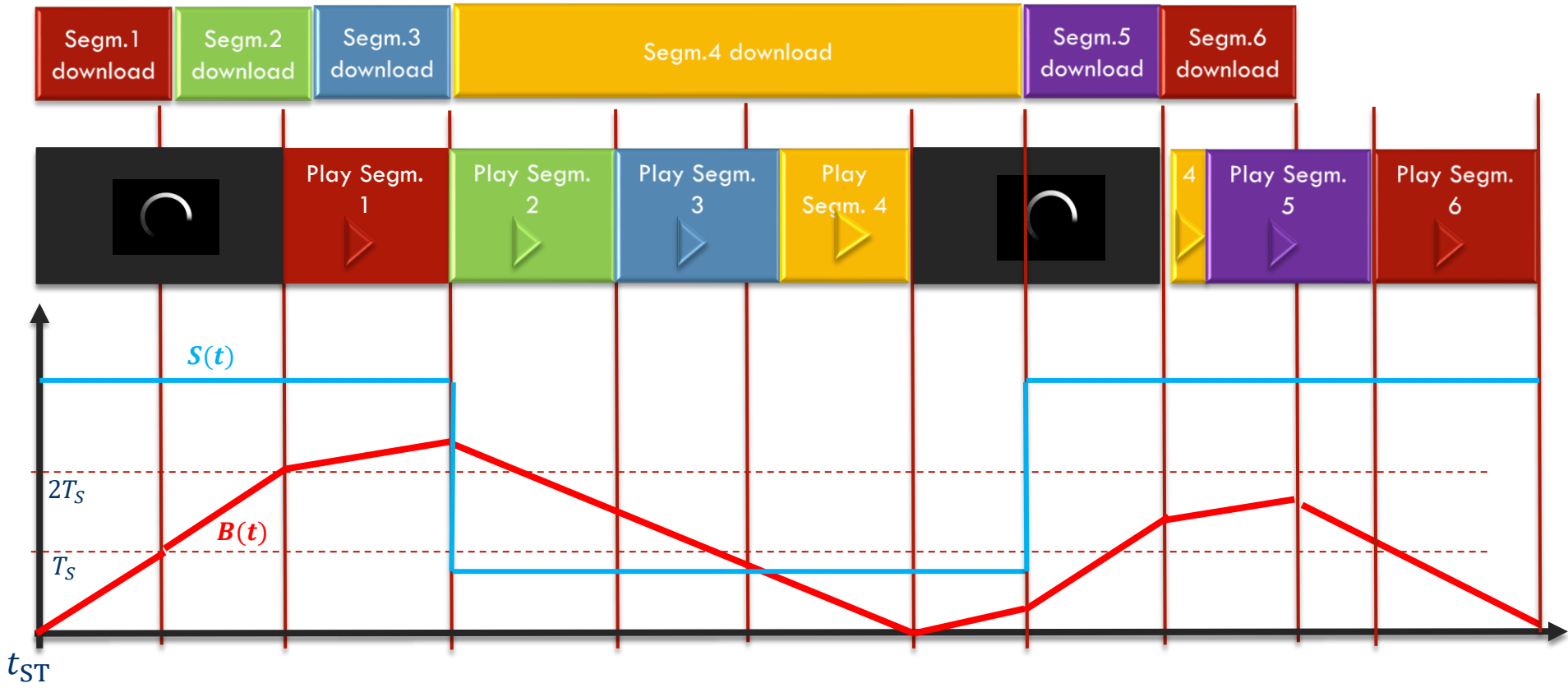
Playout Buffer Dynamics

- **Output Flow Rate $f_{\mathbf{out}}$**: The seconds of video content consumed per real-time second
- During active playback $f_{\text{out}} = 1$
  - For simplicity, we do not consider speed variations
- During rebuffering, no video is played, thus $f_{out} = 0$
- In conclusion

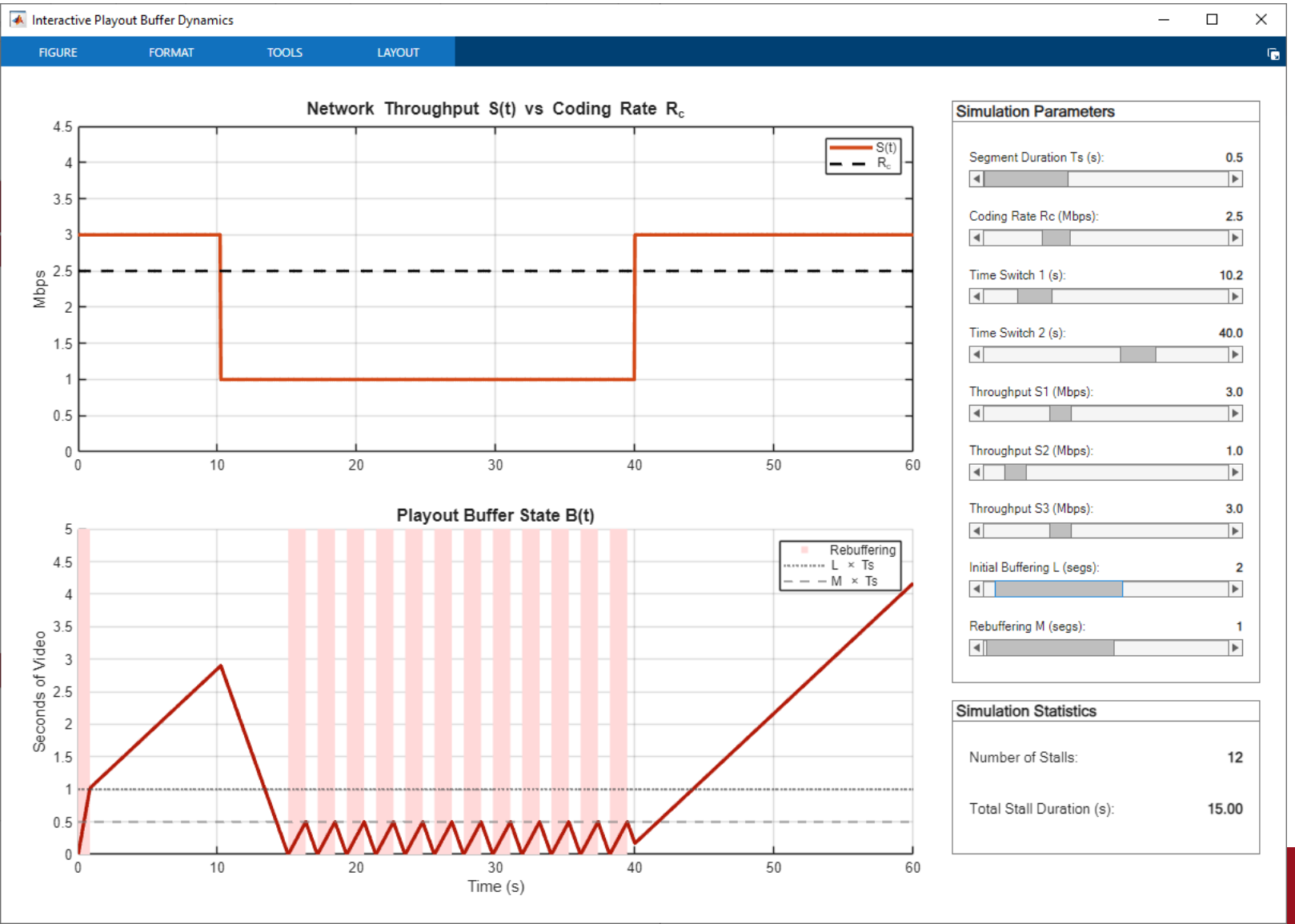
$$\frac{dB}{dt} = \begin{cases} \frac{S}{R_C} - 1 & \text{if playout} \\ \frac{S}{R_C} & \text{if rebuffering} \end{cases}$$

Examples

- Let us show what happens in a ABR system when the thoughput has a sudden drop
- $L = 2, M = 1, N = 6$



Interactive Example: Matlab simulation



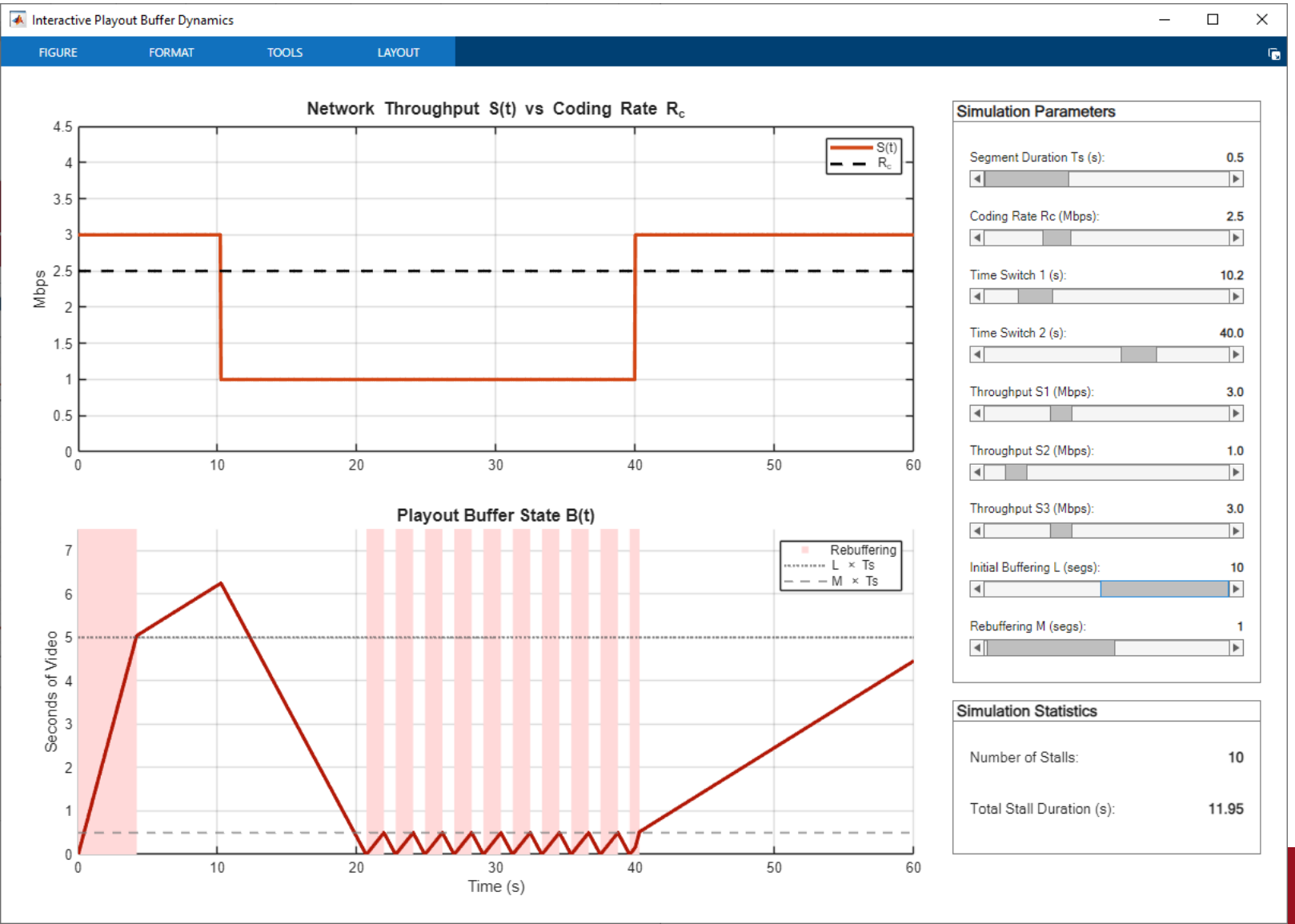
$$T_S = 0.5$$
$$L = 2$$
$$M = 1$$

Small inital buffer capacity

Fast setup

Frequent short stalls (very annoying)

Interactive Example: Matlab simulation

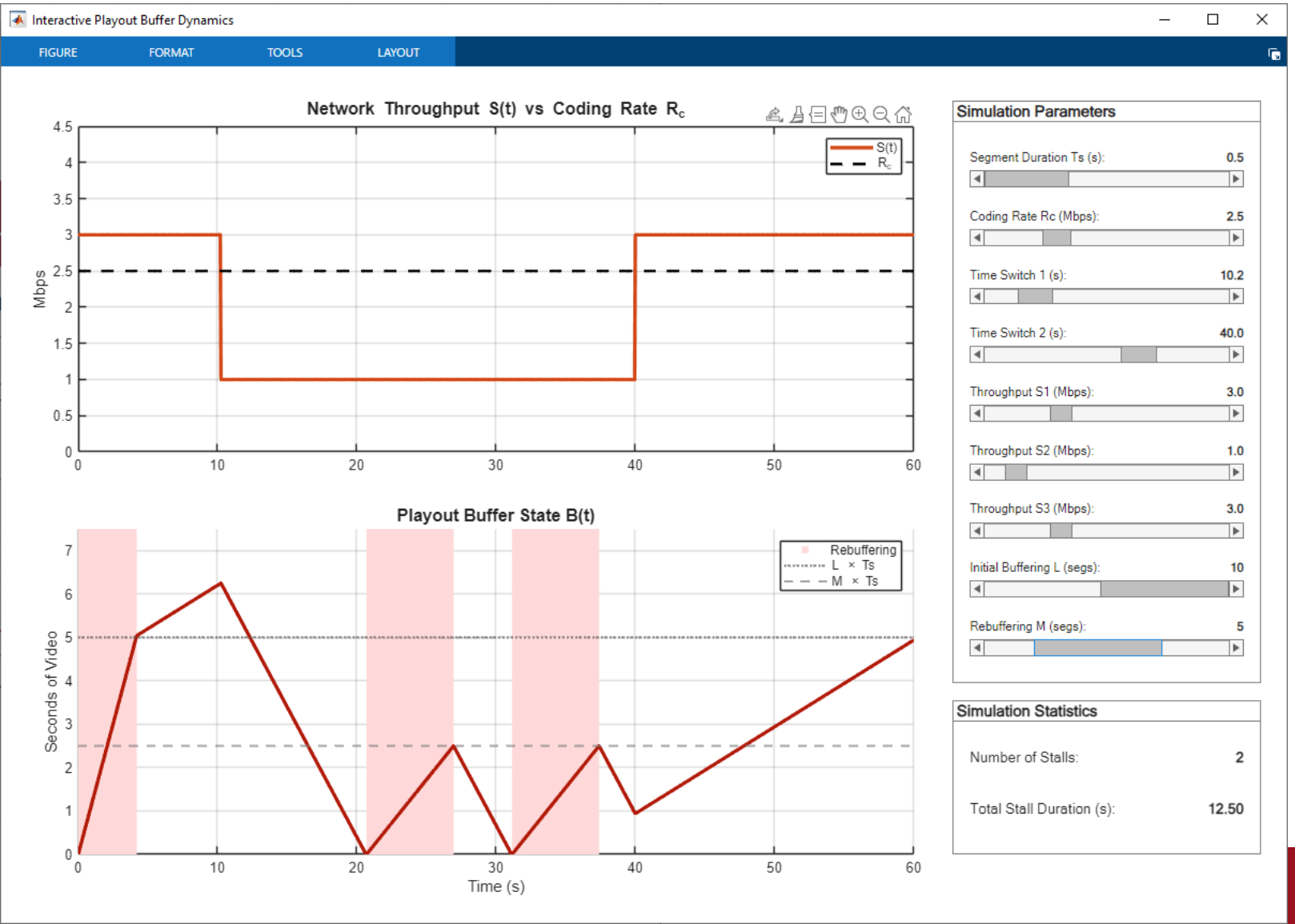


$$T_S = 0.5$$
$$L = 10$$
$$M = 1$$

Larger initial buffer capacity

Less stalls, but with the same frequency

Interactive Example: Matlab simulation



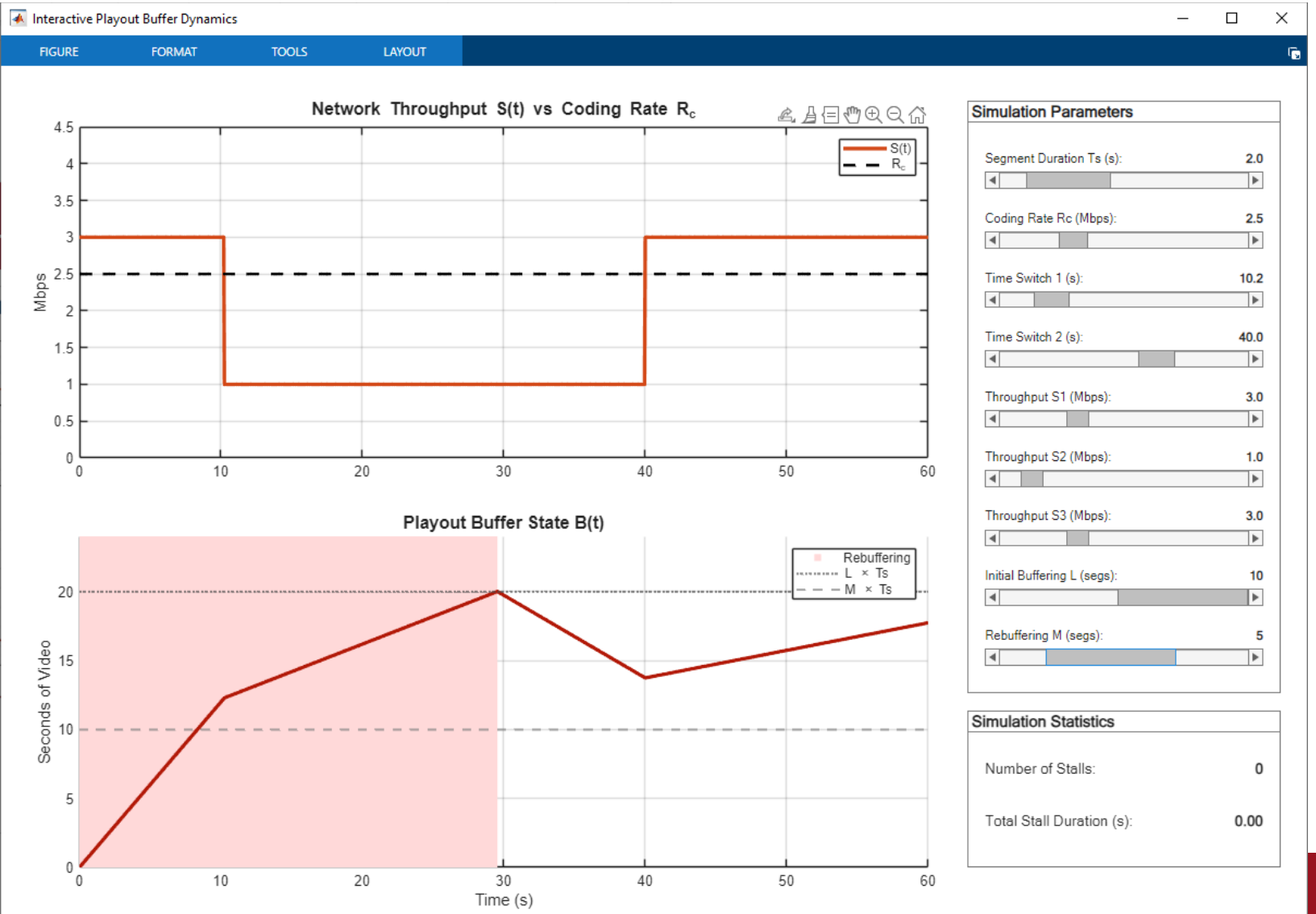
$$T_S = 0.5$$
$$L = 10$$
$$M = 5$$

Larger initial buffer capacity

Larger buffer during playout

Less stalls, but longer

Interactive Example: Matlab simulation



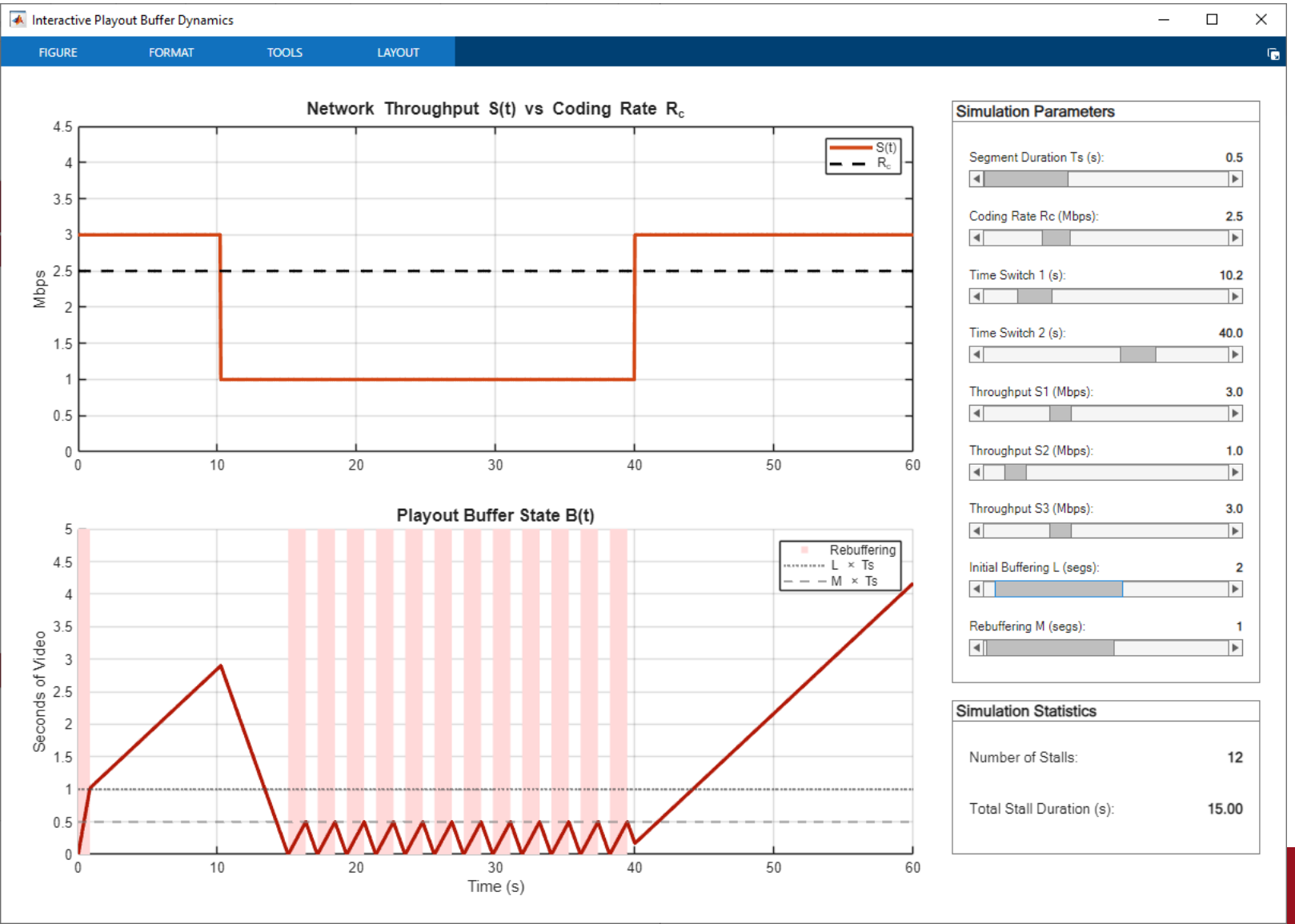
$$T_S = 2$$
$$L = 10$$
$$M = 5$$

Very large initial buffer capacity

Large buffer during playout

It absorbs completely the capacity drop

Interactive Example: Matlab simulation



$$T_S = 0.5$$
$$L = 2$$
$$M = 1$$

Wrap-up: System model for AS

The main parameters are

- T_S: playout duration of a video segment (constant)
- S_n: throughput during the download of segment n
- $q(n)$: requested level for segment n
- $T_D(n,k)$: downloading time for segment n at level k

$$T_D(n,k) = \frac{T_S R_C(k)}{S_n}$$

- $B(t)$: **the playout buffer**. At time t, $B(t)$ is the amount of video **received and not yet played.** It is measured in seconds
- The **initial buffering time** $T_{IB} = L \cdot T_S$: before starting the video playout, we fill the play-out buffer with L segments
- The rebuffering constraint during playout, $M \cdot T_S$

UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Video streaming QoE

Video Streaming Quality of Experience

- Problem: defining the quality of experience (QoE) for the streaming service

- The global QoE is influenced by:

1. The per-segment video quality
2. The number and duration of re-buffering events
3. The number and amplitude of quality changes
4. The duration of the initial buffering time

Video Streaming Quality of Experience

1. The **per-segment video quality** $Q(n)$

In general, $Q(n) = f\big(R(k_n)\big)$, where i is the number of segment and k_n is the level selected for this segment.

The function f should be a rate-quality function, like $\mathrm{MOS} = f(R)$.

However, we know that the exact behavior of the perceived quality vs. rate relationship largely depends on the content: some images could look perfect at 0.5 bpp while others show visible artifacts at 1 bpp

We also know that the value of the PSNR is only a proxy of the subjective quality

Finally, the perceived quality is, in all relevant cases, a *monotonic increasing function of the coding rate* R

In conclusion, since it is very difficult to model the relationship between MOS and coding rate, it is a common choice to just take a very simple models for f:
$Q(n) = R(k_n)$ or even $Q(n) = k_n$

Video Streaming Quality of Experience

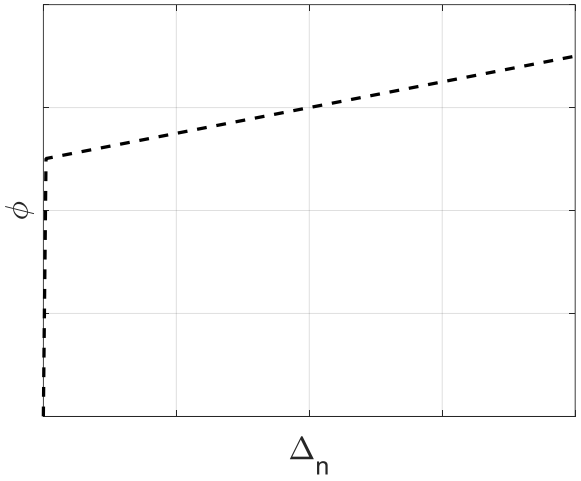
2. The **number and duration** of re-buffering events

Let us refer with Δ_n to the duration of the re-buffering event occurred during the playout of segment number n
If no rebuffering affects segment n, then $\Delta_n = 0$
Since after a rebuffering at least a full segment is downloaded before resuming the playout, there cannot be more than one rebuffering during a single segment playout.
The sheer re-buffering is very disruptive for the QoE, regardless its duration.

In conclusion, the quality of experience during segment n decreases with the duration Δ the rebuffering. The **reduction** of QoE is modeled with a function $\phi(\Delta)$, such that $\phi(0) = 0$, while $\phi(0^+)$ is relatively large but then ϕ increases slowly:

$$\phi(\Delta) = \begin{cases} 0 & \text{if } \Delta = 0 \\ a\Delta + b & \text{if } \Delta > 0 \end{cases}$$

where a is “small” and b is “large”



Video Streaming Quality of Experience

3. The **number** and **amplitude** of **quality changes**
   Any quality variation, both decrease *and increase* are annoying *per se*: it is better to have a constant, maybe somewhat low level of quality than to frequently change the level

4. The **duration** of the **initial buffering** time
   Typically, users allow an initial buffering time if it reduces the number of rebuffering later. However, a too-long initial buffering time also reduces the QoE

A video quality metric for streaming

In conclusion, the **QoE for segment n** can be represented as:

$$J(n) = \lambda_1 k_n - \lambda_2 |k_n - k_{n-1}| - \phi(\Delta_n)$$

The QoE metric $J(n)$ measured over segment n

- increases with the level k_n
- decreases if there is a level change wrt previous segment
- if there is a stall of duration Δ_n , J decreases sharply, i.e. $\phi(0) = 0$, while $\phi(0^+)$ is relatively large

- **The sequence-level QoE** is given by the sum of per-segment QoEs minus the annoyance of the initial buffering:

$$J = \sum_{n=1}^{N} J(n) - \lambda_3 T_{ST}$$

T_{ST} is the time required for downloading L segments with the quality level assigned by the streaming strategy

- It is in general difficult to set the values of λ_i's and the behavior of ϕ. λ_3 is relatively small (good tolerance to initial delay) while ϕ has a discontinuity in 0: even the smallest buffering event is considered annoying

UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Adaptive bitrate (ABR) strategies

Adaptive Bit Rate (ABR)

- A client can use a bit rate adaptation (ABR) algorithm to automatically select the “best” quality of the next segment that can be downloaded in time for playback
- ABR is typically non-standardized because is implemented only at the client’s side
- Common ABR algorithms are **throughput-based**, **buffer-based**, and **hybrid** schemes

Throughput-based ABR

- Throughput-based schemes estimate the network throughput and decide on bitrate of next segment to be downloaded
  - Choose frame quality so that estimated download time is not longer than segment playout time
  - E.g.: PANDA, Festive, and Squad

- Z. Li, X. Zhu, J. Gahm, R. Pan, H. Hu, A. C. Begen, and D. Oran, “Probe and adapt: Rate adaptation for http video streaming at scale,” IEEE Journal on Selected Areas in Communications, vol. 32, no. 4, 2014.
- J. Jiang, V. Sekar, and H. Zhang, “Improving fairness, efficiency, and stability in http-based adaptive video streaming with FESTIVE,” in Proc of Conference on Emerging Networking Experiments and Technologies. Association for Computing Machinery, 2012.

Buffer-based ABR

- Buffer-based algorithms use buffer level to decide on bitrate of next segment
  - Buffer level high/low → frames arrive faster/slower than consumed → quality can be increased/decreased
  - Such algorithms include BBA and **BOLA**

- T.-Y. Huang, R. Johari, N. McKeown, M. Trunnell, and M. Watson, "A buffer-based approach to rate adaptation: Evidence from a large video streaming service," in Proceedings of the 2014 ACM Conference on SIGCOMM. Association for Computing Machinery, 2014.
- K. Spiteri, R. Urgaonkar, and R. K. Sitaraman, "Bola: Near-optimal bitrate adaptation for online videos," in INFOCOM. IEEE, 2016.

Hybrid ABR

- Hybrid algorithms use both throughput prediction and buffer levels
  - ELASTIC, MPC, and ABMA+, DYNAMIC

  - X. Yin, A. Jindal, V. Sekar, and B. Sinopoli, “A control-theoretic approach for dynamic adaptive video streaming over http,” SIGCOMM Comput. Commun. Rev., vol. 45, no. 4, 2015.
  - Beben, Andrzej, et al. "ABMA+ lightweight and efficient algorithm for HTTP adaptive streaming." Proceedings of the 7th International Conference on Multimedia Systems. 2016.

An example of ABR (Adaptive BitRate streaming)

- **The client decides k_n with the goal of maximizing $J = \lambda_1 k_n - \lambda_2 |k_n - k_{n-1}| - \phi(\Delta_n)$**
- We need to decide the quality level for segment n, knowing the previous level k_{n-1}, the playout buffer size $B_0 > 0$ (i.e., B_0 seconds of received video have not been played yet) and an estimation of the throughput $\hat{S}$. The duration of a segment is T_S seconds

Set $J_{\min} = \infty$

For each $\ell = 1 \dots K$, (we test all the possible quality levels: we need to compute the duration of the stall Δ associated to each quality level)

Set $R = R(\ell)$: the coding rate at level k

Set $\beta = \frac{\hat{S}}{R} - 1$: it is the filling rate of the playout buffer

Assuming a constant throughput during the dwl of the segment, the buffer level is $B(t) = B_0 + \beta t$

If $\beta \geq 0$, the buffer will not reach zero during the dwl of segment n: we set $\boldsymbol{\Delta = 0}$ and go to the evaluation of J

If $\beta < 0$, the buffer starts depleting during the download of segment n

…

An example of ABR (Adaptive BitRate streaming)

If $\beta < 0$, the buffer starts depleting during the download of segment n

The buffer reaches zero at time t_0 such that $B_0 + \beta t_0 = 0$, i.e., $\boldsymbol{t_0 = \frac{B_0}{1-\frac{\hat{S}}{R}} = \frac{RB_0}{R-\hat{S}}}$

If $\mathbf{t_0 > T_S}$, the playout of the new segment will finish before the buffer is empty: $\boldsymbol{\Delta = 0}$

else ($\mathbf{t_0 < T_s}$) when the buffer reaches zero, there are $T_R = T_S - \frac{RB_0}{R-\hat{S}}$ secs of the segment to be downloaded yet: there is a stall. The duration of the stall is the time needed to download the rest of the segment plus M new segments

$$\boldsymbol{\Delta = (T_R + MT_S)R/\hat{S}}$$

Evaluate $J = \lambda_1 \ell - \lambda_2 |\ell - k_{n-1}| - \phi(\Delta)$

if $J < J_{\min}$,

set $k_n = \ell$ and $J_{\min} = J$

End for

At the end of the for loop, k_n is the best choice for the quality level (in the current hypotheses)

An example of ABR (Adaptive BitRate streaming)

This algorithm is only one among many possible ABR strategies

It has some simplistic assumptions that might undermine its behavior:

- It assumes a constant throughput during all the download time of the segment
- It also assumes a constant throughout during the rebuffering if a stall happens
- It assumes that the coding rate is constant during the playout, while we know that Intra frames have significantly higher coding rates than other frames
- There is no safeguard level of B: as long as it does not reach zero during the playout of the current segment, no action is taken to prevent stalling

Note that this algorithm only runs on the client side: each client can run a different algorithm, e.g., more «aggressive» (i.e., high quality is demanded, with higher risk of stalls) or more cautious (stall probability is made low, but the average video quality is also low)

Limits of current AS technology

- Risk of freezing/low quality if RTT is large
  - Increase in RTT makes also more difficult to estimate the throughput
- Decisions are entirely client-side
  - Client has only a partial view of the channel conditions
- Adaptation strategy may be suboptimal in the presence of multiple DASH clients or aggressive cross-traffic strategies
- Size/bitrate is not the only aspect impacting perceived quality!

UNIVERSITÀ DI PADOVA
Dipartimento
di Ingegneria
dell'Informazione

Exercises

Exercise 1

Consider an adaptive streaming system with:
Segment duration $T_S = 1$ second
Number of quality levels $K = 3$
Bit-rate per level: $R_C\ (1) = 500\ kbps,\ R_C\ (2) = 1\ Mbps,\ R_C\ (3) = 2Mbps$

The initial buffering stops after t_{ST} seconds (NB. This is the duration of initial buffering, not the amount of downloaded and playable video).

Let us set t=0 is when the first bit is received. The connection between the server and the client is characterized by the following throughput:

$\forall t \in (0, T), S(t) = S_1 = 0.4\ Mbps$

$\forall t > T, S(t) = S_2 = 0.5\ Mbps$

Question: find the minimum initial buffering that assures no stalls with $q(n) = 1$, i.e., the client always (for any n) asks for quality level 1

Solution

Solution

We observe that for $t < T, S < R_C$, while for $t > T, S = R_C$

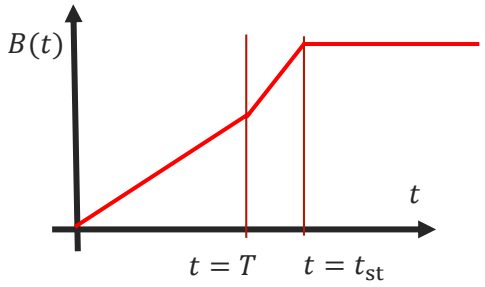
Remember that the slope of the buffer size is $B'_{\text{rebuff}}(t) = \frac{S}{R_C}$ when rebuffering and $B'_{\text{playout}}(t) = \frac{S}{R_C} - 1$ during the playout

The client is rebuffering in the interval $(0, t_{ST})$, then it starts the playout

If $\boldsymbol{t_{ST} > T}$ the slope of the buffer size is:

$$B'(t) = \begin{cases} \frac{S_1}{R_c} = 0.8 & \forall t \in (0, T) \\ \frac{S_2}{R_C} = 1 & \forall t \in (T, t_{ST}) \\ \frac{S_2}{R_C} - 1 = 0 & \forall t > t_{ST} \end{cases}$$

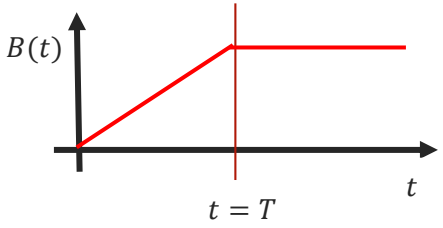
$\boldsymbol{t_{ST} > T}$



Thus, in this case the buffer never depletes (the slope is positive or null), and then there is no rebuffering.

Similarly, if $\boldsymbol{t_{ST} = T}$, the buffer size reaches $0.8T$ before the playout starts, and then it stays constant

$\boldsymbol{t_{ST} = T}$

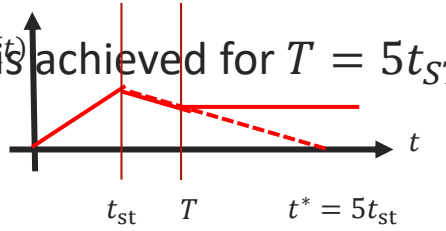


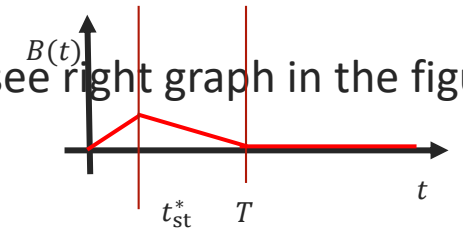
Solution

Let us now consider what happens if $t_{ST} < T$

During the interval $(0, t_{\mathrm{ST}})$ the buffer is filled with slope $B' = \frac{S_1}{R_C} = 0.8$ and then $B(t_{\mathrm{ST}}) = 0.8t_{\mathrm{ST}}$

- For $t > t_{ST}$, and until the buffer is not empty, the buffer has slope $B' = \frac{S_1}{R_C} - 1 = -0.2$ and then $B(t) = B(t_{\mathrm{ST}}) + B' \cdot (t - t_{ST}) = 0.8t_{ST} - 0.2(t - t_{\mathrm{ST}}) = t_{\mathrm{ST}} - 0.2t$
- Then the buffer size decreases linearly with time, until it reaches zero (rebuffering event) or until $t = T$, when its slope becomes zero. We compute the time the buffer reaches zero and we set t_{ST} such that it is larger than T
- $B(t^*) = 0 \Leftrightarrow t_{ST} - 0.2t^* = 0 \Leftrightarrow t^* = 5t_{ST}$
- Then if $T < 5t_{ST}$ the buffer does not empty before the change of slope, and there is no rebuffering
- The minimum initial buffering is achieved for $T = 5t_{ST}$, ie $\boldsymbol{t_{ST} = \frac{1}{5}T}$, see right graph in the figure.

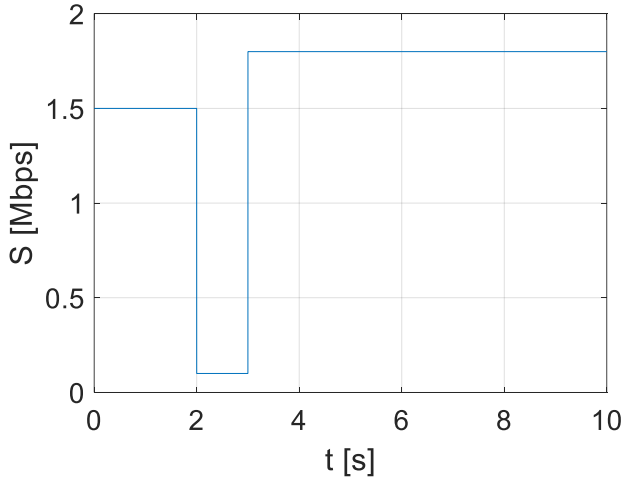




Exercise 2

An adaptive streaming system is characterized by:
Segment duration $T_S = 1$ second
Number of quality levels $K = 3$
Bit-rate per level:
$RC(1) = 500$ kbps, $RC2) = 1$ Mbps, $RC(3) = 2$Mbps
The initial buffering stops after downloading $L = 2$ segments. After a rebuffering, the playout resumes when the next segment is loaded ($M = 1$)
Let us set t=0 when the first bit is received. The following throughput characterizes the connection between the server and the client :

$$S(t) = \begin{cases} S_1 = 1.5\ Mbps & \forall t \in (0,2) \\ S_2 = 0.1\ Mbps & \forall t \in (2,3) \\ S_3 = 1.8\ Mbps & \forall t > 3 \end{cases}$$



Questions 1 and 2: If the client always requests the max quality ($q(n)$=3 $\forall n \geq 1$) when will the playout begin? And what is the frequency and duration of the rebuffering, if any?
Question 3: Show that selecting the quality according to:

$$q: n \geq 1 \rightarrow \begin{cases} 3 & \text{if } n \bmod 5 \neq 3 \\ 2 & \text{if } n \bmod 5 = 3 \end{cases}$$

avoids rebuffering at all
Question 4: is the previous strategy better than using $q = 2$ all the time?

Solution

Question 1

- The playout starts when $L = 2$ segments have been downloaded. The number of bits needed is: $N_b = T_S R_C(q(1)) + T_S R_C(q(2))$
- More in general, $N_b = T_S \sum_{n=1}^{L} R_C\big(q(n)\big)$
- In our case, $q(1) = q(2) = 3$, then $R_C(1) = R_C(2) = 2Mbps$. Each segments takes 2 Mbits and thus $N_b = 2 + 2 = 4Mbits$
- Let us find out how long it takes to download 4 Mbits. After the first T=2 seconds, we download $N_1 = T \cdot S_1 = 2 \cdot 1.5 = 3Mbits$
- In the next T=1 second, we download $N_2 = T \cdot S_2 = 1 \cdot 0.1\ Mbits$
- We still need to download $N_b - N_1 - N_2 = 0.9\ Mbits$
- Starting from time $t = 3$, this will take $\frac{N_b}{S_3} = \frac{0.9}{1.8} = 0.5\ s$
- **Then the playout will start at $t_{ST} = 3.5$**

Solution

Question 2

Starting from $t_{\mathrm{ST}} = 3.5$, the playout starts.
Since $R_C > S$, the buffer starts emptying. Thus, there will be a rebuffering event. We can compute the slope of the buffer between the time t_{ST} and the next rebuffering. Consider that for $t > 3$, $S(t) = S_3$ is a constant

$$B'(t) = \frac{S_3}{R_C} - 1 = \frac{1.8}{2} - 1 = -\frac{1}{10}$$

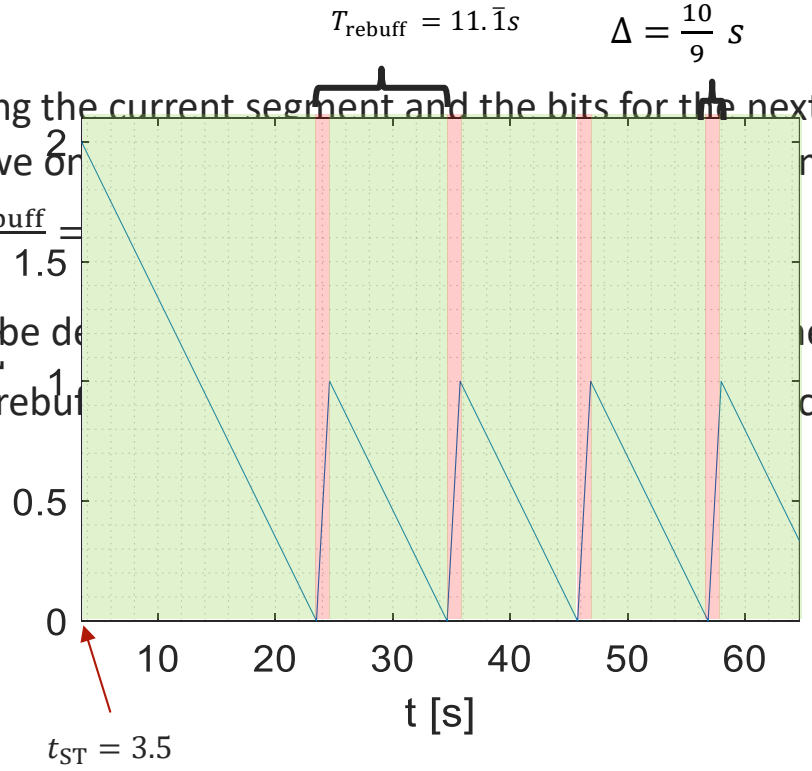
This means that the buffer shrinks of 1/10 s every second of playout. Starting from $B(t_{\mathrm{ST}}) = 2$, it will be empty after $\frac{B(t_{\mathrm{ST}})}{|B'|} = 20$ s

Recap: we have received $L = 2$ seconds (initial rebuffering) plus $20 \cdot 1.8 = 36$ Mbits during the 20 seconds of playout. This corresponds to $\frac{36}{R_C} = 18$ s of playable video. Thus, after 20 seconds of playout, we have received and played 2+18=20 seconds of video: the buffer is empty and a rebuffering event happens: $t_{\mathrm{rebuff}} = 3.5 + 20 = 23.5$

Solution

Question 2 continued

- Now the rebuffering start. It will need to download: the bits for finishing the current segment and the bits for the next segment. But the current segment (segment 20) has been completely downloaded: so, we or mber of bits needed for the rebuffering is $N_{\text{rebuff}} = T_S\ R_C(q(n)) = 2Mbits$. This takes $\frac{N_{\text{rebuff}}}{S}$ =
- After the rebuffering, the buffer has 1 second of playable video. It will be de ext rebuffering will happen after 10 seconds and will last again $\frac{10}{9}$ seconds and so on. The rebuf conds



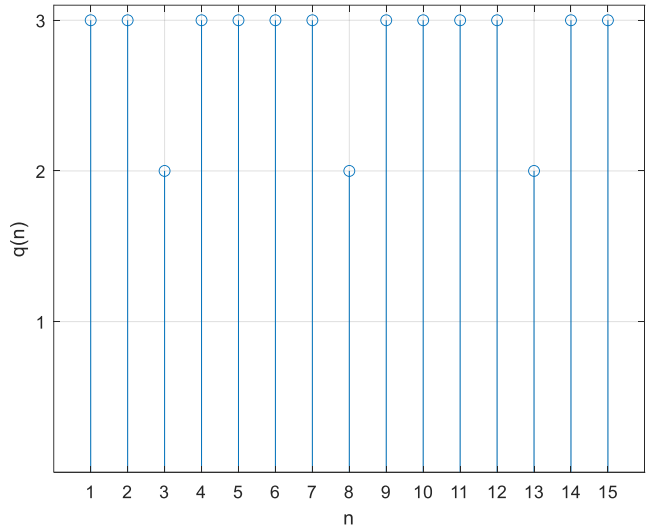
Solution

Question 3

The initial rebuffering ends at $t = 3.5$s, because the first two segments are requested with $k = 3$, exactly as in the previous case.

Thus, $B(3.5) = 2$. The third segment is requested at $k = 2$.

The download time is then: $T_D(3) = \frac{T_S R_C(2)}{S_3} = \frac{1}{1.8} s = \frac{5}{9} s$



During this time, the slope of $B(t)$ is constant and equal to

$$B'(t) = \frac{S}{R_C} - 1 = 0.8\ s/s$$

The slope is positive, so the buffer keeps filling.

After downloading the segment, the buffer reaches $B(3.5) + B'(t) \cdot T_D(3) = 2 + \frac{5}{9} \cdot \frac{4}{5} = 2.\bar{4}\ s$

Now we download 4 segments (from 4 to 7) at level $k = 3$. Since S_n is still constant, the time for downloading the four segments is 4 times the downloading of one:

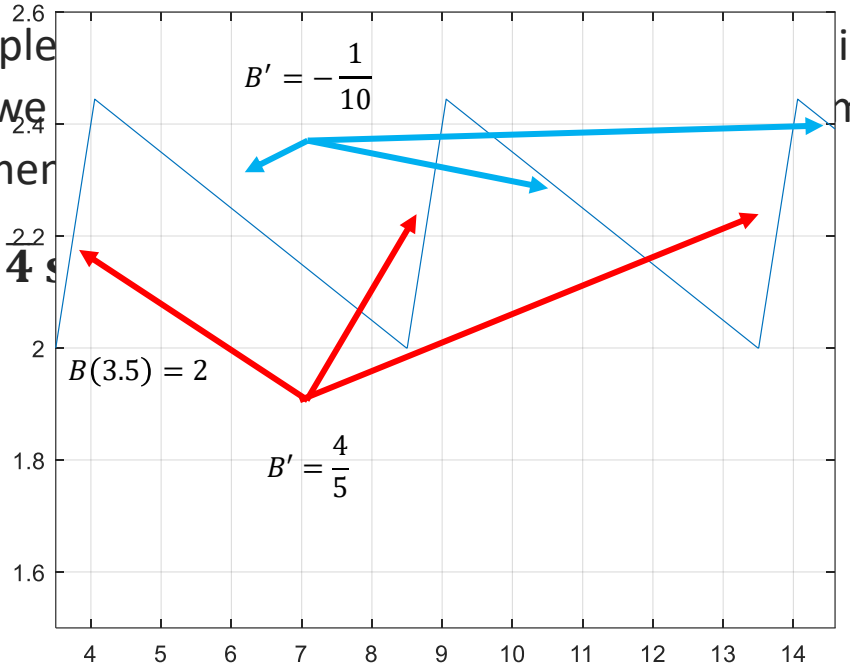
$$T_D = 4T_D(n,3) = 4\frac{T_S R_C(3)}{S_n} = 4\frac{2}{1.8} s = \frac{40}{9} s = 4.\bar{4}s$$

Notice that $R_C(3)$ refers to the quality level 3.

Solution

Question 3

- During the download of segments 4 to 7, the buffer is deple ing of segment 7 is finished, the buffer is $B(t) = 2.\overline{4} - 4.\overline{4}/10 = 2s$. Now we me $t = 3.5$, thus $B(t)$ repeats exactly as in the previous interval of 5 segmer
- In conclusion **the buffer will oscillates between 2 and 2.$\overline{4}$ s**



Exercise 2

- Question 4. Compare the QoE of the two strategies:

$$q_1 : n \geq 1 \rightarrow \begin{cases} 3 & \text{if } n \bmod 5 \neq 3 \\ 2 & \text{if } n \bmod 5 = 3 \end{cases}$$

$$q_2 : n \geq 1 \rightarrow 2 \; \forall n \geq 1$$

Using the model:

$$J = \sum_{n=1}^{N} J(n) - \lambda_3 \, T_{ST}$$

and $J(n) = \lambda_1 k_n - \lambda_2 |k_n - k_{n-1}| - \phi(\Delta_n)$

Exercise 2

For the first strategy, we have shown that there is no stall. Thus the QoE is:

$$J_1 = \sum_{n=1}^{N} [\lambda_1 q_1(n) - \lambda_2 |q_1(n) - q_1(n-1)|] - \lambda_3 T_1$$

Now, $q_1 = 3$ for $4N/5$ segments, and $|q_1(n) - q_1(n-1)| = 1$ for $2N/5$ segments. Then

$$J_1 = \frac{12}{5} N\lambda_1 + \frac{2}{5} N\lambda_1 - \frac{2}{5} N\lambda_2 - \lambda_3 L = N\left(\frac{14}{5}\lambda_1 - \frac{2}{5}\lambda_2\right) - \lambda_3 T_1$$

In the second case we have no stall neither. The QoE is

$$J_2 = \sum_{n=1}^{N} [\lambda_1 q_1(n) - \lambda_2 |q_1(n) - q_1(n-1)|] - \lambda_3 T_2 = 2N\lambda_1 - \lambda_3 T_2$$

For N large, we have $J_1 - J_2 \approx N\left(\frac{4}{5}\lambda_1 - \frac{2}{5}\lambda_2\right) > 0$ if and only if $\lambda_1 > \lambda_2/2$

Conclusion: periodically switching the quality level affects the QoE. The best strategy depends on the weigths λ which in turn are typically evaluated with subjective tests.